

beamer named overlay specifications with **beanoves**

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in **beamer** documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is a solution to this [latex.org forum query](#).

Contents

1 Installation

1.1 Package manager

When not already available, **beanoves** package may be installed using a TEX distribution's package manager, either from the graphical user interface, or with the relevant command (`tlmgr` for **TeX** Live and `mpm` for **MiKTeX**). This should install files **beanoves.sty** and its debug version **beanoves-debug.sty** as well as **beanoves-doc.pdf** documentation.

1.2 Manual installation

The **beanoves** source files are available from the [source repository](#). They can also be fetched from the [CTAN repository](#).

1.3 Usage

The **beanoves** package is imported by putting `\RequirePackage{beanoves}` in the preamble of a **LaTeX** document that uses the **beamer** class. Should the package cause problems, its features can be temporarily deactivated with simple commands `\BeanovesOff` and `\BeanovesOn`.

2 Minimal example

The L^AT_EX document below is a contrived example to show how the **beamer** overlay specifications have been extended. More demonstration files are available from the [beanoves source repository](#).

```
1 \documentclass{beamer}
2 \RequirePackage{beanoves}
3 \begin{document}
4 \Beanoves {
5   A = 1:4,
6   B = A.last::3,
7   C = B.next,
8 }
9 \begin{frame}
10  {\Large Frame \insertframenum}
11  {\Large Slide \insertslidenum}
12 -- \visible<?(A.1)> {Only on slide 1}\\
13 -- \visible<?(B.range)> {Only on slides 4 to 6}\\
14 -- \visible<?(C.1)> {Only on slide 7}\\
15 -- \visible<?(A.2)> {Only on slide 2}\\
16 -- \visible<?(B.2:B.last)> {Only on slides 5 to 6}\\
17 -- \visible<?(C.2)> {Only on slide 8}\\
18 -- \visible<?(A.next)-> {From slide 5}\\
19 -- \visible<?(B.3:B.last)> {Only on slide 6}\\
20 -- \visible<?(C.3)> {Only on slide 9}\\
21 \end{frame}
22 \end{document}
```

On line 4, we use the `\Beanoves` command to declare *named overlay sets*. On line 5, we declare an overlay set named ‘A’, which is a range starting at slide 1 and ending at slide 4. On line 12, the extended *named overlay specification* `?(A.1)` stands for 1 because 1 is the first index of the overlay set named A. On line 15, `?(A.2)` stands for 2 whereas on line 18, `?(A.next)` stands for 5. On line 6, we declare a second overlay set named ‘B’, starting after the 3 slides of ‘A’ namely 4. Its length is 3 meaning that its last slide number is 6, thus each `?(B.last)` is replaced by 6. The next slide number after slide range ‘B’ is 7 which is also the start of the third slide range due to line 7.

3 Named overlay sets

3.1 Presentation

Within a **beamer** frame, there are different slides that appear in turn according to overlay specifications. The main overlay set is a range of integers covering all the slide numbers, from one to the total amount of slides. In general, an overlay set is a range of positive integers identified by a unique name. The main practical interest is that such sets may be defined relative to one another, we can even have lists of overlay sets. Finally, we can use these lists to build and organize **beamer** overlay specifications logically.

3.2 Named overlay reference

A.1, C.2 are *named overlay references*, as well as A and X!A.B.C.2. More precisely, they are string identifiers, each one referencing a well defined static integer or list of ranges to be used in beamer overlay specifications. They have 2 components:

1. the *frame identifier* $\langle id \rangle$ before the exclamation mark !, like X, in X!A.B.C.2, optional
2. the *path* $\langle c_1 \rangle \dots \langle c_j \rangle$ like A.B.C, required ($j > 0$).

The *full path* is $\langle id \rangle! \langle c_1 \rangle \dots \langle c_j \rangle$. The *frame ids* and path components $\langle c \rangle$'s are alphanumeric case sensitive identifiers, with possible underscores but with no space. Unicode symbols above U+00A0 are allowed if the underlying T_EX engine supports it. Only the *frame id* is allowed to be empty, in which case it may apply to any common frame. The *path components* must not consist of only lowercase letters¹.

The mapping from *named overlay references* to sets of integers is defined at the global T_EX level to allow its use in `\begin{frame}<...>` and to share the same overlay sets between different frames. Hence the *frame id* due to the need to possibly target a particular frame.

3.3 Defining named overlay sets

In order to define *named overlay sets*, we can either execute the next `\Beanoves` command before a beamer frame environment, or use the custom `beanoves` option of this environment.

`\Beanoves` `\Beanoves{\langle ref_1 \rangle=\langle spec_1 \rangle, \dots, \langle ref_j \rangle=\langle spec_j \rangle}`

`\Beanoves*` Each $\langle ref \rangle$ key is a *named overlay reference* whereas each $\langle spec \rangle$ is an *overlay set specifier*. When the same $\langle ref \rangle$ key is used multiple times, only the last one is taken into account.

When performed at the document level, the `\Beanoves` command starts by cleaning what was set by previous calls. When performed inside L^AT_EX environments, each new call cumulates with the previous one. Notice that the argument of this function can contain macros: they will be exhaustively expanded at resolution time².

`beanoves` `beanoves = {\langle ref_1 \rangle=\langle spec_1 \rangle, \dots, \langle ref_j \rangle=\langle spec_j \rangle}`

The `\Beanoves` arguments take precedence over both the `\Beanoves*` arguments and the `beanoves` options. This allows to provide an overlay name only when not already defined, which is helpfull when the very same frame source is included multiple times in different contexts. Notice that $\langle ref \rangle=1$ can be shortened to $\langle ref \rangle$.

3.3.1 Value specifiers

Hereafter $\langle value \rangle$ denotes a numerical expression.

¹This will avoid collisions with the `fp` module of `expl3`.

²Precision is needed about the exact time when the expansion occurs.

Standalone

$\langle \text{ref} \rangle = \langle \text{value} \rangle$, a *value specifier* for a single number. When $= \langle \text{value} \rangle$ is omitted it defaults to $=1$.

The numerical expressions are evaluated and then rounded using `\fp_eval:n`. They can contain mathematical functions and *named overlay references* defined above. If this reference concerns a set of values, only the first one is considered.

The corresponding overlay set can be seen as a *value counter*.

3.3.2 Colon delimited range specifiers

In beamer, integer ranges are denoted by $\langle \text{first} \rangle - \langle \text{last} \rangle$. In order to authorize algebraic expressions for $\langle \text{first} \rangle$ and $\langle \text{last} \rangle$, the dash character $-$ is replaced by a colon $:$ in beanoes. Hereafter $\langle \text{first} \rangle$, $\langle \text{last} \rangle$ and $\langle \text{length} \rangle$ are placeholders for *value specifiers*.

Standalone

$\langle \text{ref} \rangle = \langle \text{first} \rangle :$,
 $\langle \text{ref} \rangle = \langle \text{first} \rangle ::$, for the infinite range of signed integers starting at and including $\langle \text{first} \rangle$.

$\langle \text{ref} \rangle = \langle \text{first} \rangle : \langle \text{last} \rangle$,
 $\langle \text{ref} \rangle = \langle \text{first} \rangle :: \langle \text{length} \rangle$,
 $\langle \text{ref} \rangle = : \langle \text{last} \rangle :: \langle \text{length} \rangle$,
 $\langle \text{ref} \rangle = :: \langle \text{length} \rangle : \langle \text{last} \rangle$, are variants for the same finite range of signed integers starting at and including $\langle \text{first} \rangle$, ending at and including $\langle \text{last} \rangle$, provided $\langle \text{first} \rangle + \langle \text{length} \rangle = \langle \text{last} \rangle + 1$. $\langle \text{first} \rangle$ can be omitted, in which case it defaults to 1. Additionally $: \langle \text{last} \rangle$ and $:: \langle \text{length} \rangle$ are then equivalent.

$\langle \text{ref} \rangle = : \langle \text{last} \rangle$,
 $\langle \text{ref} \rangle = :: \langle \text{length} \rangle$, are syntactic sugar when $\langle \text{first} \rangle$ is 1.

3.3.3 List specifiers

$\langle \text{ref} \rangle = [\langle \text{def}_1 \rangle, \dots, \langle \text{def}_j \rangle]$, where $\langle \text{def}_k \rangle$, $1 \leq k \leq j$, is one of

- $\langle \text{index} \rangle = \langle \text{value} \rangle$,
- $\langle \text{value} \rangle$, a shortcut for $\langle i \rangle = \langle \text{value} \rangle$, $\langle i \rangle$ being the smallest positive integer such that $\langle \text{ref} \rangle . \langle i \rangle$ is not already defined.

The first step is to remove previous $\langle \text{ref} \rangle$ related definitions, then execute the various $\langle \text{ref} \rangle . \langle i \rangle = \langle \text{value} \rangle$ definitions in the order given. Here is the **implementation**.

$\langle \text{ref} \rangle = \{ \langle \text{def}_1 \rangle, \dots, \langle \text{def}_j \rangle \}$, where $\langle \text{def}_k \rangle$, $1 \leq k \leq j$, is one of

- $\langle \text{name} \rangle = \langle \text{spec} \rangle$,
- $\langle \text{name} \rangle$ for $\langle \text{name} \rangle = 1$,

The first step is to remove previous $\langle \text{ref} \rangle$ related definitions, then execute the various $\langle \text{ref} \rangle . \langle \text{name} \rangle = \langle \text{spec} \rangle$ definitions in the order given. In a final step, the $\langle \text{name} \rangle$'s are collected in a comma sepated list to initialize $\langle \text{ref} \rangle$ with. $\langle \text{spec} \rangle$ is any specifier. Here is the **implementation**.

4 Resolution of $\langle \dots \rangle$ query expressions

This is the key feature of the `beanoves` package, extending `beamer overlay specifications` normally included between pointed brackets. Before the *overlay specifications* are processed by the `beamer` class, the `beanoves` package scans them for any occurrence of $\langle \langle \text{queries} \rangle \rangle$. Each one is then evaluated and replaced by its resolved static counterpart. The overall result is finally forwarded to the `beamer` class.

The $\langle \text{queries} \rangle$ argument is a comma separated list of individual $\langle \text{query} \rangle$'s processed from left to right as explained below. Notice that nesting a $\langle \dots \rangle$ query expression inside another query expression is supported, embedded expressions being resolved first.

The named overlay sets defined above are queried for integer numerical values that will be passed to `beamer`. Turning an *overlay query* into the static expression it represents, as when above $\langle A.1 \rangle$ was replaced by 1, is denoted by *overlay query resolution* or simply *resolution*. The process starts by replacing any *query reference* by its value as explained below until obtaining numerical expressions that are evaluated and finally rounded to the nearest integer to feed `beamer` with either ranges or numbers. When the *query reference* is a previously declared $\langle \text{ref} \rangle$, like **A** after **A=1**, it is simply replaced by the corresponding declared $\langle \text{value} \rangle$, here 1. Otherwise, we use *implicit overlay queries* and their *resolution rules* depending on the definition of the named overlay set. Hereafter $\langle i \rangle$ denotes a signed integer whereas $\langle \text{value} \rangle$, $\langle \text{first} \rangle$, $\langle \text{last} \rangle$, $\langle \text{length} \rangle$ and $\langle i \text{ expr} \rangle$ stand for raw integers or more general numerical expressions that are evaluated beforehand.

Notice that *resolution* occurs only when requested and the result is cached for performance reason. Hereafter, we start by default resolution and we give the resolution of user queries after named overlay sets are defined.

4.1 Range overlay queries

$\langle \text{ref} \rangle = \langle \text{first} \rangle$: as well as $\langle \text{first} \rangle ::$ define a range limited from below:

overlay query	resolution
$\langle \text{ref} \rangle$	$\langle \text{first} \rangle -$
$\langle \text{ref} \rangle.1$	$\langle \text{first} \rangle$
$\langle \text{ref} \rangle.2$	$\langle \text{first} \rangle + 1$
$\langle \text{ref} \rangle.\langle i \rangle$	$\langle \text{first} \rangle + \langle i \rangle - 1$
$\langle \text{ref} \rangle[\langle i \text{ expr} \rangle]$	$\langle \text{first} \rangle + \langle i \text{ expr} \rangle - 1$
$\langle \text{ref} \rangle.\text{previous}$	$\langle \text{first} \rangle - 1$
$\langle \text{ref} \rangle.\text{first}$	$\langle \text{first} \rangle$

Notice that $\langle \text{ref} \rangle.\text{previous}$ and $\langle \text{ref} \rangle.0$ are most of the time synonyms.

$\langle \text{ref} \rangle = \langle \text{first} \rangle : \langle \text{last} \rangle$ as well as variants $\langle \text{first} \rangle :: \langle \text{length} \rangle$, $:: \langle \text{length} \rangle : \langle \text{last} \rangle$ or $: \langle \text{last} \rangle :: \langle \text{length} \rangle$, which are equivalent provided $\langle \text{first} \rangle + \langle \text{length} \rangle = \langle \text{last} \rangle + 1$.

Define a range limited from both above and below:

overlay query	resolution
$\langle \text{ref} \rangle$	$\langle \text{first} \rangle - \langle \text{last} \rangle$
$\langle \text{ref} \rangle.1$	$\langle \text{first} \rangle$
$\langle \text{ref} \rangle.2$	$\langle \text{first} \rangle + 1$
$\langle \text{ref} \rangle.\langle i \rangle$	$\langle \text{first} \rangle + \langle i \rangle - 1$
$\langle \text{ref} \rangle[\langle i \text{ expr} \rangle]$	$\langle \text{first} \rangle + \langle i \text{ expr} \rangle - 1$
$\langle \text{ref} \rangle.\text{previous}$	$\langle \text{first} \rangle - 1$
$\langle \text{ref} \rangle.\text{first}$	$\langle \text{first} \rangle$
$\langle \text{ref} \rangle.\text{last}$	$\langle \text{last} \rangle$
$\langle \text{ref} \rangle.\text{next}$	$\langle \text{last} \rangle + 1$
$\langle \text{ref} \rangle.\text{length}$	$\langle \text{length} \rangle$

Notice that the resolution of the $\langle \text{ref} \rangle$ overlay query is a beamer range and not an algebraic difference, negative integers do not make sense there while in `beamer` context.

In the frame example below, we use the `\BeanovesResolve` command for the demonstration. It is mainly used for debugging and testing purposes.

```

1 \Beanoves {
2   A = 3:8, % or similarly A = 3::6, A = ::6:8 and A = :8::6
3 }
4 \begin{frame} {Frame \insertframenum} {Slide \insertslidenumber}
5 \ttfamily
6 \BeanovesResolve[show] (A)          == 3-8,
7 \BeanovesResolve[show] (A.1)       == 3,
8 \BeanovesResolve[show] (A.-1)      == 1,
9 \BeanovesResolve[show] (A.previous) == 2,
10 \BeanovesResolve[show] (A.first)   == 3,
11 \BeanovesResolve[show] (A.last)    == 8,
12 \BeanovesResolve[show] (A.next)    == 9,
13 \BeanovesResolve[show] (A.length)  == 6,
14 \end{frame}

```

$\langle \text{ref} \rangle = [\dots]$

$\langle \text{ref} \rangle = \{\dots\}$ See the list range specifiers in section 3.3.3.

4.2 Value counter queries

$\langle \text{ref} \rangle = \langle \text{value} \rangle$ defines a counter value.

overlay query	resolution
$\langle \text{ref} \rangle$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.1$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.2$	$\langle \text{value} \rangle + 1$
$\langle \text{ref} \rangle.\langle i \rangle$	$\langle \text{value} \rangle + \langle i \rangle - 1$
$\langle \text{ref} \rangle[\langle i \text{ expr} \rangle]$	$\langle \text{value} \rangle + \langle i \text{ expr} \rangle - 1$
$\langle \text{ref} \rangle.\text{previous}$	$\langle \text{value} \rangle - 1$
$\langle \text{ref} \rangle.\text{first}$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.\text{last}$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.\text{next}$	$\langle \text{value} \rangle + 1$

Additionally, resolution rules are provided for dedicated *overlay queries*. Here, $\langle ref \rangle$ is considered a standard programming variable:

$\langle ref \rangle = \langle set\ expression \rangle$, resolve $\langle set\ expression \rangle$, assign the result to the $\langle ref \rangle$ and use it. It defines $\langle ref \rangle$ globally if not already done. Here $\langle set\ expression \rangle$ is the longest character sequence with no space³. To remove this limitation, use an embedded query expression or enclose the `clist` item within a pair of braces.

$\langle ref \rangle += \langle integer\ expression \rangle$, resolve $\langle integer\ expression \rangle$ into $\langle integer \rangle$, advance $\langle ref \rangle$ by $\langle integer \rangle$ and use the result.

$++\langle ref \rangle$, increment $\langle ref \rangle$ by 1 and use it.

$\langle ref \rangle ++$, use $\langle ref \rangle$ and then increment it by 1.

This can be used for an indirection.

IN PROGRESS

```

1 \Beanoves {
2   A = 1,
3   B = [ 10 = 100 ],
4   C = 10,
5 }
6 \begin{frame} {Frame \insertframenum} {Slide \insertslidenumber}
7 \ttfamily
8 \BeanovesResolve[show] (A.C)      == \BeanovesResolve[show] (A.10) == 10,
9 \BeanovesResolve[show] (B.C)      == \BeanovesResolve[show] (B.10) == 100,
10 \BeanovesResolve[show] (A.C+=10) == \BeanovesResolve[show] (A.20) == 20,
11 \BeanovesResolve[show] (A.C)      == \BeanovesResolve[show] (A.20) == 20,
12 \BeanovesResolve[show] (A.C+=10) == \BeanovesResolve[show] (A.20) == 20+10,
13 \BeanovesResolve[show] (A.C)      == \BeanovesResolve[show] (A.20) == 30,
14 \BeanovesResolve[show] (B.C)      == \BeanovesResolve[show] (B.20) == 20,
15 \end{frame}

```

In order to decrement a counter, one can increment with a negative value, no dedicated syntax is provided yet.

For each new frame, these counters are reset to the value they were initialized with. Sometimes, resetting the counter manually is necessary, for example when managing `tikz` overlay material.

`\BeanovesReset` `\BeanovesReset` [*options*] [$\langle ref_1 \rangle [= \langle spec_1 \rangle]$, ..., $\langle ref_j \rangle [= \langle spec_j \rangle]$]

This command is very similar to `\Beanoves`, except that a standalone $\langle ref_i \rangle$ resets the counter to the last value it was initialized with, and that it is meant to be used inside a `frame` environment. Each $= \langle spec_j \rangle$ is optional and defaults to `=1`. When the `all` option is provided, some internals that were cached for performance reasons are cleared as well.

4.3 The beamer counters

While inside a `frame` environment, it is possible to save the current value of the `beamerpauses` counter that controls whether elements should appear on the current slide. For that, we can execute one of `\Beanoves{\langle ref \rangle=pauses}` or within a query

³The parser for algebraic expression is very rudimentary.

`?(\dots?(\ref)=pauses)\dots)`. Then later on, we can use `?(\dots\ref)\dots)` to refer to this saved value in the same frame⁴. Next frame source is an example of usage.

```

1 \begin{frame}
2 \visible<+>{A}\\
3 \visible<+>{B\Beanoves{afterB=pauses}}\\
4 \visible<+>{C}\\
5 \visible<?(afterB)>{other C}\\
6 \visible<?(afterB.previous)>{other B}\\
7 \end{frame}

```

“A” first appears on slide 1, “B” on slide 2 and “C” on slide 3. On line 2, `afterB` takes the value of the `beamerpauses` counter once updated, *id est* 3. “B” and “other B” as well as “C” and “other C” appear at the same time. If the `beamerpauses` counter is not suitable, we can execute instead one of `\Beanoves{(\ref)=slideinframe}` or inside a query `?(\dots)(\ref)=slideinframe)\dots)`. It uses the numerical value of `\insertslideinframe`.

4.4 Multiple queries

It is possible to replace the comma separated list of queries `?(\langle query_1 \rangle), \dots, ?(\langle query_j \rangle)` with the shorter single query `?(\langle query_1 \rangle), \dots, \langle query_j \rangle)`.

4.5 Overriding the default resolution

After `\Beanoves{A = 3:8}`, `A.first` is resolved into 3. With a supplemental instruction `\Beanoves{A.first = 10}`, `A.first` is then resolved into 10 whereas `A.1` is still resolved into 3.

4.6 Frame id

Except for very special situations, the *frame ids* can be left unspecified. When no *frame id* was explicitly provided, `beanoves` uses the *last frame id* and, if the resolution fails, an empty *frame id*. At the beginning of each frame, the *last frame id* is set to the *frame id* of the current frame, which is denoted *current frame id* and is empty by default. Then it gets updated after each named reference resolution where a *frame id* is explicitly given. For example, the first time `A.1` reference is resolved within a given frame, it is first translated to `\last frame id!A.1`, but when used just after `Y!C.2`, for example, it becomes a shortcut to `Y!A.1` because the *last frame id* is then `Y`.

In order to set the *frame id* of the current frame to frame `\langle id \rangle`, use the new `beanoves` id option of the `beamer` frame environment.

`beanoves id` `beanoves id=\langle frame id \rangle,`

We can use the same `\langle frame id \rangle` for different frames to share named overlay sets. Another possibility offered by the `beanoves` package to share named overlay sets is a fall back mechanism, for example when `X!A` cannot be resolved, `!A` is resolved instead.

⁴See [stackexchange](#) for an alternative that needs at least two passes.

4.7 Resolution command

`\BeanovesResolve` `\BeanovesResolve` [`setup`] `{queries}`

This function resolves the `queries`, which are like the argument of `?(...)` instructions: a comma separated list of single `query`'s. It is mainly used for debugging purposes. The optional `setup` is a key-value list:

`show` the result is left into the input stream

`in:N=command` the result is stored into `command`.

5 Support

See the [source repository](#). One can report issues there.

6 Implementation

Identify the internal prefix (L^AT_EX3 DocStrip convention).

1 `@@=bnvs`

6.1 Package declarations

```

2 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
3 \ProvidesExplPackage
4   {beanoves}
5   {2025/06/03}
6   {1.0}
7   {Named overlay specifications for beamer}
```

6.2 Overview

Reserved namespace: identifiers containing the case insensitive string `beanoves` or containing the case insensitive string `bnvs` delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into `beamer` specifications based on this data model.

The development is made easier due to a facility layer over `expl`.

6.3 Facility layer: definitions and naming

In order to make the code shorter and easier to read during development, we add a layer over L^AT_EX3. The `c` and `v` argument specifiers take a slightly different meaning when used in a function which name contains `bnvs` or `BNVS`. Where L^AT_EX3 would transform `l__bnvs_ref_tl` into `\l__bnvs_ref_tl`, `bnvs` will directly transform `ref` into `\l__bnvs_ref_tl`. The type of the local variable used depends on the context and may be `seq` or `int` for example. There are however a pair of exceptions mentionned below. For a better reading experience, “`ref`” will generally stand for `\l__bnvs_ref_tl`, whereas

“path sequence” will generally stand for `\l__bnvs_path_seq`. Other similar shortcuts are used as well.

Functions with BNVS in their names are management functions. They belong to a deeper layer and do not really contain any logic specific to the `beanoves` package.

```
\BNVS:c      \BNVS:c {<cs core name>}
\BNVS_l:cn   \BNVS_l:cn {<local variable core name>} {<type>}
\BNVS_g:cn   \BNVS_g:cn {<global variable core name>} {<type>}
```

These are naming functions internally used to focus on the discriminating part of variable or function names.

```
8 \cs_new:Npn \BNVS:c      #1 { __bnvs_#1 }
9 \cs_new:Npn \BNVS_l:cn #1 #2 { l__bnvs_#1_#2 }
10 \cs_new:Npn \BNVS_g:cn #1 #2 { g__bnvs_#1_#2 }
```

```
\BNVS_use_raw:N \BNVS_use_raw:N {<cs>}
\BNVS_use_raw:c \BNVS_use_raw:c {<cs name>}
\BNVS_use_raw:Nc \BNVS_use_raw:Nc {<function>} {<cs name>}
\BNVS_use_raw:nc \BNVS_use_raw:nc {<tokens>} {<cs name>}
\BNVS_use:c      \BNVS_use:c {<cs core>}
\BNVS_use:Nc     \BNVS_use:Nc {<function>} {<cs core>}
\BNVS_use:nc     \BNVS_use:nc {<tokens>} {<cs core>}
```

`\BNVS_use_raw:c` is a convenient wrapper over `\use:c`. possibly prepended with some code, for debugging and testing. It needs 3 expansion steps just like `\BNVS_use:c`. The other are used to expand `\use:c` enough before usage by `<function>` or `<tokens>`. The first argument of `<function>` has type N. The next token after `<tokens>` will have type N too. `<cs name>` is a full cs name whereas `<cs core>` will be prepended with the appropriate prefix specific to the `beanoves` package.

```
11 \cs_new:Npn \BNVS_use_raw:N #1 { #1 }
12 \cs_new:Npn \BNVS_use_raw:c #1 {
13   \exp_last_unbraced:No
14   \BNVS_use_raw:N { \cs:w #1 \cs_end: }
15 }

16 \cs_new:Npn \BNVS_use:c #1 {
17   \BNVS_use_raw:c { \BNVS:c { #1 } }
18 }

19 \cs_new:Npn \BNVS_use_raw:NN #1 #2 {
20   #1 #2
21 }

22 \cs_new:Npn \BNVS_use_raw:nN #1 #2 {
23   #1 #2
24 }

25 \cs_new:Npn \BNVS_use_raw:Nc #1 #2 {
26   \exp_args:NNc \BNVS_use_raw:NN #1 { #2 }
27 }
```

```

28 \cs_new:Npn \BNVS_use_raw:nc #1 #2 {
29   \exp_last_unbraced:Nno
30   \BNVS_use_raw:nN { #1 } { \cs:w #2 \cs_end: }
31 }

32 \cs_new:Npn \BNVS_use:Nc #1 #2 {
33   \BNVS_use_raw:Nc #1 { \BNVS:c { #2 } }
34 }

35 \cs_new:Npn \BNVS_use:nc #1 #2 {
36   \BNVS_use_raw:nc { #1 } { \BNVS:c { #2 } }
37 }

38 \tl_new:N \l__bnvs_last_unbraced_tl
39 \cs_new:Npn \BNVS_tl_last_unbraced:nv #1 {
40   \tl_set:Nn \l__bnvs_last_unbraced_tl { #1 }
41   \BNVS_tl_use:nc { \exp_last_unbraced:NV \l__bnvs_last_unbraced_tl }
42 }

43 \cs_new:Npn \BNVS_tl_use:nvv #1 #2 {
44   \BNVS_tl_use:nv { \BNVS_tl_use:nv { #1 } { #2 } }
45 }

46 \cs_new:Npn \BNVS_tl_use:nvvv #1 #2 {
47   \BNVS_tl_use:nvv { \BNVS_tl_use:nv { #1 } { #2 } }
48 }

49 \cs_new:Npn \BNVS_tl_use:nvvvv #1 #2 {
50   \BNVS_tl_use:nvvv { \BNVS_tl_use:nv { #1 } { #2 } }
51 }

52 \cs_new:Npn \BNVS_log:n #1 { }
53 \cs_generate_variant:Nn \BNVS_log:n { x }

```

6.3.1 Debug

<code>\BNVS_DEBUG_ON:</code>	<code>\BNVS_DEBUG_ON:</code>
<code>\BNVS_DEBUG_OFF:</code>	<code>\BNVS_DEBUG_OFF:</code>
<code>\BNVS_DEBUG_push:nn</code>	<code>\BNVS_DEBUG_push:nn {<types on>} {<types off>}</code>
<code>\BNVS_DEBUG_pop:</code>	<code>\BNVS_DEBUG_pop:</code>

These functions activate or deactivate the debug mode. They are only active in the `beanoves-debug` package. Manage debug messaging for one given `<type>` or `<types>`. The implementation is not publicly exposed. The `<type>` is a single letter or `**`.

6.3.2 Facility layer: Functions

<code>\BNVS_new:cpn</code>	<code>\BNVS_new:cpn</code> is like <code>\cs_new:cpn</code> except that the name argument is tagged for <code>beanoves</code>
<code>\BNVS_set:cpn</code>	package. Similarly for <code>\BNVS_set:cpn</code> .

```

54 \cs_new:Npn \BNVS_new:cpn #1 {
55   \cs_new:cpn { \BNVS:c { #1 } }
56 }

```

```

57 \cs_new:Npn \BNVS_set:cpn #1 {
58   \cs_set:cpn { \BNVS:c { #1 } }
59 }

60 \cs_generate_variant:Nn \cs_generate_variant:Nn { c }
61 \cs_new:Npn \BNVS_generate_variant:cn #1 {
62   \cs_generate_variant:cn { \BNVS:c { #1 } }
63 }

```

6.3.3 logging

<pre> \BNVS_warning:n \BNVS_warning:x \BNVS_error:n \BNVS_error:x \BNVS_fatal:n \BNVS_fatal:x </pre>	<pre> \BNVS_warning:n {<message>} \BNVS_error:n {<message>} \BNVS_fatal:n {<message>} </pre> Very rudimentary. No long message available for error recovery.
--	---

```

64 \msg_new:nnn { beanoves } { :n } { #1 }
65 \msg_new:nnn { beanoves } { :nn } { #1~(#2) }

66 \cs_new:Npn \BNVS_warning:n {
67   \msg_warning:nnn { beanoves } { :n }
68 }
69 \cs_new:Npn \BNVS_warning:x {
70   \msg_warning:nnx { beanoves } { :n }
71 }

72 \cs_new:Npn \BNVS_error:n {
73   \msg_error:nnn { beanoves } { :n }
74 }
75 \cs_generate_variant:Nn \BNVS_error:n { x }

76 \cs_new:Npn \BNVS_fatal:n {
77   \msg_fatal:nnn { beanoves } { :n }
78 }
79 \cs_new:Npn \BNVS_fatal:x {
80   \msg_fatal:nnx { beanoves } { :n }
81 }
82 \cs_new:Npn \BNVS_fatal_unreachable: {
83   \BNVS_fatal:n { Unreachable }
84 }
85 \cs_new:Npn \BNVS_fatal_unreachable: { }

```

6.3.4 Facility layer: Variables

```
\BNVS_N_new:c \BNVS_N_new:c {<type>}
\BNVS_v_new:c \BNVS_v_new:c {<type>}
```

Creates a collection of typed utility functions, see usage below. These functions are undefined when no longer used. `<type>` is one of `tl`, `seq...`

```

86 \cs_new:Npn \BNVS_N_new:c #1 {
87   \cs_new:cpn { BNVS_#1:c } ##1 {
88     1 \BNVS:c{ ##1 } \tl_if_empty:nF { ##1 } { _ } #1
89   }
90   \cs_new:cpn { BNVS_#1_new:c } ##1 {
91     \use:c { #1_new:c } { \use:c { BNVS_#1:c } { ##1 } }
92   }
93   \cs_new:cpn { BNVS_#1_use:c } ##1 {
94     \use:c { \cs:w BNVS_#1:c \cs_end: { ##1 } }
95   }
96   \cs_new:cpn { BNVS_#1_use:Nc } ##1 ##2 {
97     \BNVS_use_raw:Nc
98     ##1 { \cs:w BNVS_#1:c \cs_end: { ##2 } }
99   }
100  \cs_new:cpn { BNVS_#1_use:nc } ##1 ##2 {
101    \BNVS_use_raw:nc
102    { ##1 } { \cs:w BNVS_#1:c \cs_end: { ##2 } }
103  }
104 }

105 \cs_new:Npn \BNVS_v_new:c #1 {
106   \cs_new:cpn { BNVS_#1_use:Nv } ##1 ##2 {
107     \exp_args:Nv ##1
108     { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
109   }
110   \cs_new:cpn { BNVS_#1_use:cv } ##1 ##2 {
111     \exp_args:Nnv \BNVS_use:c { ##1 }
112     { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
113   }
114   \cs_new:cpn { BNVS_#1_use:nv } ##1 ##2 {
115     \exp_args:Nnv \use:n { ##1 }
116     { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
117   }
118 }

119 \BNVS_N_new:c { bool }
120 \BNVS_N_new:c { int }
121 \BNVS_v_new:c { int }
122 \BNVS_N_new:c { tl }
123 \BNVS_v_new:c { tl }
124 \cs_new:Npn \BNVS_tl_use:Nvv #1 #2 {
125   \BNVS_tl_use:nv { \BNVS_tl_use:Nv #1 { #2 } }
126 }
```

```

127 \cs_new:Npn \BNVS_tl_use:Nvvv #1 #2 {
128   \BNVS_tl_use:nvv { \BNVS_tl_use:Nv #1 { #2 } }
129 }

130 \cs_new:Npn \BNVS_tl_use:Nvvvv #1 #2 {
131   \BNVS_tl_use:nvvv { \BNVS_tl_use:Nv #1 { #2 } }
132 }

133 \BNVS_N_new:c { str }
134 \BNVS_v_new:c { str }
135 \BNVS_N_new:c { seq }
136 \BNVS_v_new:c { seq }
137 \cs_undefine:N \BNVS_N_new:c

```

`\BNVS_use:Ncn` `\BNVS_use:Ncn` *<function>* *{<core name>}* *{<type>}*

```

138 \cs_new:Npn \BNVS_use:Ncn #1 #2 #3 {
139   \BNVS_use_raw:c { BNVS_#3_use:Nc }   #1   { #2 }
140 }

141 \cs_new:Npn \BNVS_use:ncn #1 #2 #3 {
142   \BNVS_use_raw:c { BNVS_#3_use:nc } { #1 } { #2 }
143 }

144 \cs_new:Npn \BNVS_use:Nvn #1 #2 #3 {
145   \BNVS_use_raw:c { BNVS_#3_use:Nv }   #1   { #2 }
146 }

147 \cs_new:Npn \BNVS_use:nvn #1 #2 #3 {
148   \BNVS_use_raw:c { BNVS_#3_use:nv } { #1 } { #2 }
149 }

150 \cs_new:Npn \BNVS_use:Ncn cn #1 #2 #3 {
151   \BNVS_use:ncn {
152     \BNVS_use:Ncn   #1   { #2 } { #3 }
153   }
154 }

155 \cs_new:Npn \BNVS_use:ncn cn #1 #2 #3 {
156   \BNVS_use:ncn {
157     \BNVS_use:ncn { #1 } { #2 } { #3 }
158   }
159 }

160 \cs_new:Npn \BNVS_use:Nvncn #1 #2 #3 {
161   \BNVS_use:ncn {
162     \BNVS_use:Nvn   #1   { #2 } { #3 }
163   }
164 }

165 \cs_new:Npn \BNVS_use:nvncn #1 #2 #3 {
166   \BNVS_use:ncn {
167     \BNVS_use:nvn { #1 } { #2 } { #3 }
168   }
169 }

```

```

170 \cs_new:Npn \BNVS_use:Ncncncn #1 #2 #3 #4 #5 {
171   \BNVS_use:ncn {
172     \BNVS_use:Ncncn   #1   { #2 } { #3 } { #4 } { #5 }
173   }
174 }

175 \cs_new:Npn \BNVS_use:ncncncn #1 #2 #3 #4 #5 {
176   \BNVS_use:ncn {
177     \BNVS_use:ncncn { #1 } { #2 } { #3 } { #4 } { #5 }
178   }
179 }

```

\BNVS_new_c:cn \BNVS_new_c:nc {<type>} {<core name>}

```

180 \cs_new:Npn \BNVS_new_c:nc #1 #2 {
181   \BNVS_new:cpn { #1_#2:c } {
182     \BNVS_use_raw:c { BNVS_#1_use:nc } { \BNVS_use_raw:c { #1_#2:N } }
183   }
184 }

185 \cs_new:Npn \BNVS_new_cn:nc #1 #2 {
186   \BNVS_new:cpn { #1_#2:cn } ##1 {
187     \BNVS_use:ncn { \BNVS_use_raw:c { #1_#2:Nn } } { ##1 } { #1 }
188   }
189 }

190 \cs_new:Npn \BNVS_new_cnn:ncN #1 #2 #3 {
191   \BNVS_new:cpn { #2:cnn } ##1 {
192     \BNVS_use:Ncn { #3 } { ##1 } { #1 }
193   }
194 }

195 \cs_new:Npn \BNVS_new_cnn:nc #1 #2 {
196   \BNVS_use_raw:nc {
197     \BNVS_new_cnn:ncN { #1 } { #1_#2 }
198   } { #1_#2:Nnn }
199 }

200 \cs_new:Npn \BNVS_new_cnv:ncN #1 #2 #3 {
201   \BNVS_new:cpn { #2:cnv } ##1 ##2 {
202     \BNVS_tl_use:nv {
203       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
204     }
205   }
206 }

207 \cs_new:Npn \BNVS_new_cnv:nc #1 #2 {
208   \BNVS_use_raw:nc {
209     \BNVS_new_cnv:ncN { #1 } { #1_#2 }
210   } { #1_#2:Nnn }
211 }

```

```

212 \cs_new:Npn \BNVS_new_cnx:ncN #1 #2 #3 {
213   \BNVS_new_cpn { #2:cnx } ##1 ##2 {
214     \exp_args:Nnx \use:n {
215       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
216     }
217   }
218 }

219 \cs_new:Npn \BNVS_new_cnx:nc #1 #2 {
220   \BNVS_use_raw:nc {
221     \BNVS_new_cnx:ncN { #1 } { #1_#2 }
222   } { #1_#2:Nnn }
223 }

224 \cs_new:Npn \BNVS_new_cc:ncNn #1 #2 #3 #4 {
225   \BNVS_new_cpn { #2:cc } ##1 ##2 {
226     \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 } { #4 }
227   }
228 }

229 \cs_new:Npn \BNVS_new_cc:ncn #1 #2 {
230   \BNVS_use_raw:nc {
231     \BNVS_new_cc:ncNn { #1 } { #1_#2 }
232   } { #1_#2:NN }
233 }

234 \cs_new:Npn \BNVS_new_cc:nc #1 #2 {
235   \BNVS_new_cc:ncn { #1 } { #2 } { #1 }
236 }

237 \cs_new:Npn \BNVS_new_cn:ncNn #1 #2 #3 #4 {
238   \BNVS_new_cpn { #2:cn } ##1 {
239     \BNVS_use:Ncn #3 { ##1 } { #1 }
240   }
241 }

242 \cs_new:Npn \BNVS_new_cn:ncn #1 #2 {
243   \BNVS_use_raw:nc {
244     \BNVS_new_cn:ncNn { #1 } { #1_#2 }
245   } { #1_#2:Nn }
246 }

247 \cs_new:Npn \BNVS_new_cv:ncNn #1 #2 #3 #4 {
248   \BNVS_new_cpn { #2:cv } ##1 ##2 {
249     \BNVS_use:nvn {
250       \BNVS_use:Ncn #3 { ##1 } { #1 }
251     } { ##2 } { #4 }
252   }
253 }

254 \cs_new:Npn \BNVS_new_cv:ncn #1 #2 {
255   \BNVS_use_raw:nc {
256     \BNVS_new_cv:ncNn { #1 } { #1_#2 }
257   } { #1_#2:Nn }
258 }

```

```

259 \cs_new:Npn \BNVS_new_cv:nc #1 #2 {
260   \BNVS_new_cv:ncn { #1 } { #2 } { #1 }
261 }

262 \cs_new:Npn \BNVS_l_use:Ncn #1 #2 #3 {
263   \BNVS_use_raw:Nc #1 { \BNVS_l:cn { #2 } { #3 } }
264 }

265 \cs_new:Npn \BNVS_l_use:ncn #1 #2 #3 {
266   \BNVS_use_raw:nc { #1 } { \BNVS_l:cn { #2 } { #3 } }
267 }

268 \cs_new:Npn \BNVS_g_use:Ncn #1 #2 #3 {
269   \BNVS_use_raw:Nc #1 { \BNVS_g:cn { #2 } { #3 } }
270 }

271 \cs_new:Npn \BNVS_g_use:ncn #1 #2 #3 {
272   \BNVS_use_raw:nc { #1 } { \BNVS_g:cn { #2 } { #3 } }
273 }

```

`\BNVS_new_conditional:cpnn` `\BNVS_new_conditional:cpnn {<core>} {<parameter> {<conditions>} {<code>}}`

```

274 \cs_generate_variant:Nn \prg_new_conditional:Npnn { c }

275 \cs_new:Npn \BNVS_new_conditional:cpnn #1 {
276   \prg_new_conditional:cpnn { \BNVS:c { #1 } }
277 }
278 \cs_generate_variant:Nn \prg_generate_conditional_variant:Nnn { c }
279 \cs_new:Npn \BNVS_generate_conditional_variant:cnn #1 {
280   \prg_generate_conditional_variant:cnn { \BNVS:c { #1 } }
281 }

282 \cs_new:Npn \BNVS_new_conditional_vn:cNnn #1 #2 #3 #4 {
283   \BNVS_new_conditional:cpnn { #1:vn } ##1 ##2 { #4 } {
284     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
285       \prg_return_true:
286     } {
287       \prg_return_false:
288     }
289   }
290 }

291 \cs_new:Npn \BNVS_new_conditional_vn:cnn #1 #2 {
292   \BNVS_use:nc {
293     \BNVS_new_conditional_vn:cNnn { #1 }
294   } { #1:nn TF } { #2 }
295 }

296 \cs_new:Npn \BNVS_new_conditional_vc:cNnn #1 #2 #3 #4 {
297   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #4 } {
298     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
299       \prg_return_true:
300     } {
301       \prg_return_false:
302     }
303   }
304 }

```

```

305 \cs_new:Npn \BNVS_new_conditional_vc:cnn #1 {
306   \BNVS_use:nc {
307     \BNVS_new_conditional_vc:cNnn { #1 }
308   } { #1:ncTF }
309 }

310 \cs_new:Npn \BNVS_new_conditional_vvc:cNnnn #1 #2 #3 #4 #5 {
311   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #5 } {
312     \BNVS_use:nvn {
313       \BNVS_use:Nvn #2 { ##1 } { #3 }
314     } { ##2 } { #4 } { ##3 } {
315       \prg_return_true:
316     } {
317       \prg_return_false:
318     }
319   }
320 }

321 \cs_new:Npn \BNVS_new_conditional_vvc:cnnn #1 {
322   \BNVS_use:nc {
323     \BNVS_new_conditional_vvc:cNnnn { #1 }
324   } { #1:nncTF }
325 }

326 \cs_new:Npn \BNVS_new_conditional_vc:cNn #1 #2 #3 {
327   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #3 } {
328     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } {
329       \prg_return_true:
330     } {
331       \prg_return_false:
332     }
333   }
334 }

335 \cs_new:Npn \BNVS_new_conditional_vc:cn #1 {
336   \BNVS_use:nc {
337     \BNVS_new_conditional_vc:cNn { #1 }
338   } { #1:ncTF }
339 }

340 \cs_new:Npn \BNVS_new_conditional_vvc:cNn #1 #2 #3 {
341   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #3 } {
342     \BNVS_tl_use:nv {
343       \BNVS_tl_use:Nv #2 { ##1 }
344     } { ##2 } { ##3 } {
345       \prg_return_true:
346     } {
347       \prg_return_false:
348     }
349   }
350 }

```

```

351 \cs_new:Npn \BNVS_new_conditional_vvc:cn #1 {
352   \BNVS_use:nc {
353     \BNVS_new_conditional_vvc:cNn { #1 }
354   } { #1:nncTF }
355 }

```

6.3.5 Regex

```

356 \cs_new:Npn \BNVS_regex_use:Nc #1 #2 {
357   \BNVS_use_raw:Nc #1 { c \BNVS:c { #2 } _regex }
358 }

```

6.3.6 Token lists

<u>__bnvs_tl_clear:c</u>	__bnvs_tl_clear:c {<core key tl>}
__bnvs_tl_clear_new:c	__bnvs_tl_use:c {<core>}
__bnvs_tl_use:c	__bnvs_tl_count:c {<core>}
__bnvs_tl_set_eq:cc	__bnvs_tl_set_eq:cc {<lhs core name>} {<rhs core name>}
__bnvs_tl_set:cn	__bnvs_tl_set:cn {<core>} {<tl>}
__bnvs_tl_set:(cv cx ce co)	__bnvs_tl_set:cv {<core>} {<value core name>}
__bnvs_tl_put_left:cn	__bnvs_tl_put_left:cn {<core>} {<tl>}
__bnvs_tl_put_right:cn	__bnvs_tl_put_right:cn {<core>} {<tl>}
__bnvs_tl_put_right:(cx ce cv co)	__bnvs_tl_put_right:cv {<core>} {<value core name>}
	__bnvs_tl_put_right:co {<core>} {<tl>}

These are shortcuts to

- \tl_clear:c {l__bnvs_<core>_tl}
- \tl_clear_new:c {l__bnvs_<core>_tl}
- \tl_use:c {l__bnvs_<core>_tl}
- \tl_set_eq:cc {l__bnvs_<lhs core>_tl} {l__bnvs_<rhs core>_tl}
- \tl_set:cv {l__bnvs_<core>_tl} {l__bnvs_<value core>_tl}
- \tl_set:cx {l__bnvs_<core>_tl} {<tl>}
- \tl_set:co {l__bnvs_<core>_tl} {<tl>}
- \tl_put_left:cn {l__bnvs_<core>_tl} {<tl>}
- \tl_put_right:cn {l__bnvs_<core>_tl} {<tl>}
- \tl_put_right:cv {l__bnvs_<core>_tl} {l__bnvs_<value core>_tl}
- \tl_put_right:co {l__bnvs_<core>_tl} {<tl>}

__bnvs_tl_clear_all:n __bnvs_tl_clear_all:n {<cores>}

Run __bnvs_tl_clear:c for each individual item of <cores>.

```

359 \BNVS_new:cpn { tl_clear_all:n } #1 {
360   \tl_map_inline:nn { #1 } {
361     \__bnvs_tl_clear:c { ##1 }
362   }
363 }

```

`\BNVS_new_conditional_vnc:cn` `\BNVS_new_conditional_vnc:cn {<core>} {<conditions>}`

Creates `<core>:vncTF` from `<core>:nncTF`.

```

364 \cs_new:Npn \BNVS_new_conditional_vnc:cNn #1 #2 #3 {
365   \BNVS_new_conditional:cpnn { #1:vnc } ##1 ##2 ##3 { #3 } {
366     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } { ##3 } {
367       \prg_return_true:
368     } {
369       \prg_return_false:
370     }
371   }
372 }

373 \cs_new:Npn \BNVS_new_conditional_vnc:cn #1 {
374   \BNVS_use:nc {
375     \BNVS_new_conditional_vnc:cNn { #1 }
376   } { #1:nncTF }
377 }

```

`\BNVS_new_conditional_vvnc:cn` `\BNVS_new_conditional_vvnc:cn {<core>} {<conditions>}`

Creates `<core>:vvncTF` from `<core>:nnncTF`.

```

378 \cs_new:Npn \BNVS_new_conditional_vvnc:cNn #1 #2 #3 {
379   \BNVS_new_conditional:cpnn { #1:vvnc } ##1 ##2 ##3 ##4 { #3 } {
380     \BNVS_tl_use:nv {
381       \BNVS_tl_use:Nv #2 { ##1 }
382     } { ##2 } { ##3 } { ##4 } {
383       \prg_return_true:
384     } {
385       \prg_return_false:
386     }
387   }
388 }

389 \cs_new:Npn \BNVS_new_conditional_vvnc:cn #1 {
390   \BNVS_use:nc {
391     \BNVS_new_conditional_vvnc:cNn { #1 }
392   } { #1:nnncTF }
393 }

```

```

394 \cs_new:Npn \BNVS_new_conditional_vvvc:cNn #1 #2 #3 {
395   \BNVS_new_conditional:cpnn { #1:vvvc } ##1 ##2 ##3 ##4 { #3 } {
396     \BNVS_tl_use:nvv {
397       \BNVS_tl_use:Nv #2 { ##1 }
398     } { ##2 } { ##3 } { ##4 } {
399       \prg_return_true:
400     } {
401       \prg_return_false:
402     }
403   }
404 }

```

`\BNVS_new_conditional_vvvc:cn` `\BNVS_new_conditional_vvvc:cn {<core>} {<conditions>}`

Creates `<core>:vvvcTF` from `<core>:nnncTF`.

```

405 \cs_new:Npn \BNVS_new_conditional_vvvc:cn #1 {
406   \BNVS_use:nc {
407     \BNVS_new_conditional_vvvc:cNn { #1 }
408   } { #1:nnncTF }
409 }

410 \cs_new:Npn \BNVS_new_tl_c:c {
411   \BNVS_new_c:nc { tl }
412 }
413 \BNVS_new_tl_c:c { clear }
414 \BNVS_new_tl_c:c { clear_new }
415 \BNVS_new_tl_c:c { use }
416 \BNVS_new_tl_c:c { count }

417 \BNVS_new:cpn { tl_set:eq:cc } #1 #2 {
418   \BNVS_use:ncncn { \tl_set:eq:NN } { #1 } { tl } { #2 } { tl }
419 }

420 \cs_new:Npn \BNVS_new_tl_cn:c {
421   \BNVS_new_cn:nc { tl }
422 }

423 \cs_new:Npn \BNVS_new_tl_cv:c #1 {
424   \BNVS_new_cv:ncn { tl } { #1 } { tl }
425 }
426 \BNVS_new_tl_cn:c { set }
427 \BNVS_new_tl_cv:c { set }

428 \BNVS_new:cpn { tl_set:cx } {
429   \exp_args:Nnx \__bnvs_tl_set:cn
430 }

431 \BNVS_new:cpn { tl_set:ce } {
432   \exp_args:Nne \__bnvs_tl_set:cn
433 }

434 \BNVS_new:cpn { tl_set:co } {
435   \exp_args:Nno \__bnvs_tl_set:cn
436 }

```

```

437 \BNVS_new_tl_cn:c { put_right }
438 \BNVS_new_tl_cv:c { put_right }
439 % \BNVS_generate_variant:cn { tl_put_right:cn } { cx }

440 \BNVS_new:cpn { tl_put_right:cx } {
441   \exp_args:Nnnx \BNVS_use:c { tl_put_right:cn }
442 }

443 \BNVS_new:cpn { tl_put_right:ce } {
444   \exp_args:Nnne \BNVS_use:c { tl_put_right:cn }
445 }

446 \BNVS_new:cpn { tl_put_right:co } {
447   \exp_args:Nnno \BNVS_use:c { tl_put_right:cn }
448 }

449 \BNVS_new_tl_cn:c { put_left }
450 \BNVS_new_tl_cv:c { put_left }
451 % \BNVS_generate_variant:cn { tl_put_left:cn } { cx }
452 % \begin{BNVS.test}{bnvs:c={ tl_put_right:co }, ignore}
453 % \__bnvs_tl_set:cn { TEST_A } { JES }
454 % \__bnvs_tl_set:co { TEST_B } { { \l__bnvs_TEST_A_tl } }
455 % \assert_equal_tl:vn { TEST_B } { { JES } } A
456 % \end{BNVS.test}

457 \BNVS_new:cpn { tl_put_left:cx } {
458   \exp_args:Nnnx \BNVS_use:c { tl_put_left:cn }
459 }

```

```

\__bnvs_tl_if_exist:cTF \__bnvs_tl_if_empty:cTF  {\langle core\rangle} {\langle yes:\rangle} {\langle no:\rangle}
\__bnvs_tl_if_empty:cTF \__bnvs_tl_if_exist:cTF  {\langle core\rangle} {\langle yes:\rangle} {\langle no:\rangle}
\__bnvs_tl_if_blank:vTF \__bnvs_tl_if_blank:vTF  {\langle core\rangle} {\langle yes:\rangle} {\langle no:\rangle}
\__bnvs_tl_if_eq:cnTF \__bnvs_tl_if_eq:cnTF  {\langle core\rangle} {\langle tl\rangle} {\langle yes:\rangle} {\langle no:\rangle}

```

These are shortcuts to

- \tl_if_exist:cTF {l__bnvs_<core>_tl} {\langle yes:\rangle} {\langle no:\rangle}
- \tl_if_empty:cTF {l__bnvs_<core>_tl} {\langle yes:\rangle} {\langle no:\rangle}
- \tl_if_eq:cnTF {l__bnvs_<core>_tl}{\langle tl\rangle} {\langle yes:\rangle} {\langle no:\rangle}

```

460 \cs_new:Npn \BNVS_new_conditional_c:ncNn #1 #2 #3 #4 {
461   \BNVS_new_conditional:cpnn { #2 } ##1 { #4 } {
462     \BNVS_use:Ncn #3 { ##1 } { #1 } {
463       \prg_return_true:
464     } {
465       \prg_return_false:
466     }
467   }
468 }

```

```

469 \cs_new:Npn \BNVS_new_conditional_c:ncn #1 #2 {
470   \BNVS_use_raw:nc {
471     \BNVS_new_conditional_c:ncNn { #1 } { #1_#2:c }
472   } { #1_#2:Ntf }
473 }
474 \BNVS_new_conditional_c:ncn { tl } { if_empty } { p, T, F, TF }
475 \BNVS_new_conditional_c:ncn { tl } { if_exist } { p, T, F, TF }

476 \BNVS_new_conditional:cpnn { tl_if_blank:v } #1 { T, F, TF } {
477   \BNVS_tl_use:Nv \tl_if_blank:nTF { #1 } {
478     \prg_return_true:
479   } {
480     \prg_return_false:
481   }
482 }

483 \cs_new:Npn \BNVS_new_conditional_cn:ncNn #1 #2 #3 #4 {
484   \BNVS_new_conditional:cpnn { #2:cn } ##1 ##2 { #4 } {
485     \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 } {
486       \prg_return_true:
487     } {
488       \prg_return_false:
489     }
490   }
491 }

492 \cs_new:Npn \BNVS_new_conditional_cn:ncn #1 #2 {
493   \BNVS_use_raw:nc {
494     \BNVS_new_conditional_cn:ncNn { #1 } { #1_#2 }
495   } { #1_#2:NnTF }
496 }
497 \BNVS_new_conditional_cn:ncn { tl } { if_eq } { T, F, TF }

498 \cs_new:Npn \BNVS_new_conditional_cv:ncNn #1 #2 #3 #4 {
499   \BNVS_new_conditional:cpnn { #2:cv } ##1 ##2 { #4 } {
500     \BNVS_use:nvn {
501       \BNVS_use:Ncn #3 { ##1 } { #1 }
502       } { ##2 } { #1 } {
503       \prg_return_true:
504     } {
505       \prg_return_false:
506     }
507   }
508 }

509 \cs_new:Npn \BNVS_new_conditional_cv:ncn #1 #2 {
510   \BNVS_use_raw:nc {
511     \BNVS_new_conditional_cv:ncNn { #1 } { #1_#2 }
512   } { #1_#2:NnTF }
513 }
514 \BNVS_new_conditional_cv:ncn { tl } { if_eq } { T, F, TF }

515 \BNVS_new:cpn { gput_right:cccn } #1 #2 #3 {
516   \tl_gput_right:cn { \_bnvs_g_IPK:w } { #1 } { #2 } { #3 } }
517 }

```

6.3.7 Strings

`\_bnvs_str_if_eq:vvTF` `\_bnvs_str_if_eq:vvTF {<core>} {<tl>} {<yes:>} {<no:>}`

These are shortcuts to

- `\str_if_eq:ccTF` `l\_bnvs_<core>_tl` `l\_bnvs_<core>_tl {<yes:>} {<no:>}`

```

518 \cs_new:Npn \BNVS_new_conditional_vv:cnN #1 #2 #3 {
519   \BNVS_new_conditional:cpnn { #1:vv } ##1 ##2 { #3 } {
520     \BNVS_tl_use:nv {
521       \BNVS_tl_use:Nv #2 { ##1 }
522     } { ##2 } {
523       \prg_return_true:
524     } {
525       \prg_return_false:
526     }
527   }
528 }

529 \cs_new:Npn \BNVS_new_conditional_vv:cn #1 {
530   \BNVS_use:nc {
531     \BNVS_new_conditional_vvnc:cnN { #1 }
532   } { #1:nnTF }
533 }

534 \cs_new:Npn \BNVS_new_conditional_vn:cnN #1 #2 #3 #4 {
535   \BNVS_new_conditional:cpnn { #2:vn } ##1 ##2 { #4 } {
536     \BNVS_use:Nvn #3 { ##1 } { #1 } { ##2 } {
537       \prg_return_true:
538     } {
539       \prg_return_false:
540     }
541   }
542 }

543 \cs_new:Npn \BNVS_new_conditional_vn:ncn #1 #2 {
544   \BNVS_use_raw:nc {
545     \BNVS_new_conditional_vn:cnN { #1 } { #1_#2 }
546   } { #1_#2:nnTF }
547 }
548 \BNVS_new_conditional_vn:ncn { str } { if_eq } { T, F, TF }

549 \cs_new:Npn \BNVS_new_conditional_vv:cnN #1 #2 #3 #4 {
550   \BNVS_new_conditional:cpnn { #2:vv } ##1 ##2 { #4 } {
551     \BNVS_use:nvn {
552       \BNVS_use:Nvn #3 { ##1 } { #1 }
553     } { ##2 } { #1 } {
554       \prg_return_true:
555     } {
556       \prg_return_false:
557     }
558   }
559 }

```

```

560 \cs_new:Npn \BNVS_new_conditional_vv:ncn #1 #2 {
561   \BNVS_use_raw:nc {
562     \BNVS_new_conditional_vv:ncNn { #1 } { #1_#2 }
563   } { #1_#2:nnTF }
564 }
565 \BNVS_new_conditional_vv:ncn { str } { if_eq } { T, F, TF }

```

6.3.8 Sequences

<code>__bnvs_seq_count:c</code>	<code>__bnvs_seq_new:c {<core>}</code>
<code>__bnvs_seq_clear:c</code>	<code>__bnvs_seq_count:c {<core>}</code>
<code>__bnvs_seq_set_eq:cc</code>	<code>__bnvs_seq_clear:c {<core>}</code>
<code>__bnvs_seq_gset_eq:cc</code>	<code>__bnvs_seq_set_eq:cc {<core₁>} {<core₂>}</code>
<code>__bnvs_seq_use:cn</code>	<code>__bnvs_seq_use:cn {<core>} {<separator>}</code>
<code>__bnvs_seq_item:cn</code>	<code>__bnvs_seq_item:cn {<core>} {<integer expression>}</code>
<code>__bnvs_seq_remove_all:cn</code>	<code>__bnvs_seq_remove_all:cn {<core>} {<tl>}</code>
<code>__bnvs_seq_put_left:cn</code>	<code>__bnvs_seq_put_right:cn {<seq core>} {<tl>}</code>
<code>__bnvs_seq_put_left:cv</code>	<code>__bnvs_seq_put_right:cv {<seq core>} {<tl core>}</code>
<code>__bnvs_seq_put_right:cn</code>	<code>__bnvs_seq_set_split:cnn {<seq core>} {<tl>} {<separator>}</code>
<code>__bnvs_seq_put_right:cv</code>	<code>__bnvs_seq_pop_left:cc {<core₁>} {<core₂>}</code>
<code>__bnvs_seq_set_split:cnn</code>	
<code>__bnvs_seq_set_split:(cnv cnx)</code>	
<code>__bnvs_seq_pop_left:cc</code>	

These are shortcuts to

- `\seq_set_eq:cc {l__bnvs_<core1>_seq} {l__bnvs_<core2>_seq}`
- `\seq_count:c {l__bnvs_<core>_seq}`
- `\seq_use:cn {l__bnvs_<core>_seq} {<separator>}`
- `\seq_item:cn {l__bnvs_<core>_seq} {<integer expression>}`
- `\seq_remove_all:cn {l__bnvs_<core>_seq} {<tl>}`
- `\seq_clear:c {l__bnvs_<core>_seq}`
- `\seq_put_right:cv {l__bnvs_<core>_seq} {l__bnvs_<core>_tl}`
- `\seq_set_split:cnn {l__bnvs_<core>_seq} {l__bnvs_<core>_tl} \KWNmeta{tl}`

```

566 \BNVS_new_c:nc { seq } { count }
567 \BNVS_new_c:nc { seq } { clear }
568 \BNVS_new_cn:nc { seq } { use }
569 \BNVS_new_cn:nc { seq } { item }
570 \BNVS_new_cn:nc { seq } { remove_all }
571 \BNVS_new_cn:nc { seq } { map_inline }
572 \BNVS_new_cc:nc { seq } { set_eq }
573 \BNVS_new_cc:nc { seq } { gset_eq }
574 \BNVS_new_cn:ncn { seq } { put_left } { tl }
575 \BNVS_new_cv:ncn { seq } { put_left } { tl }

```

```

576 \BNVS_new_cn:ncn { seq } { put_right } { t1 }
577 \BNVS_new_cv:ncn { seq } { put_right } { t1 }
578 \BNVS_new_cnn:nc { seq } { set_split }
579 \BNVS_new_cnv:nc { seq } { set_split }
580 \BNVS_new_cnx:nc { seq } { set_split }
581 \BNVS_new_cc:ncn { seq } { pop_left } { t1 }
582 \BNVS_new_cc:ncn { seq } { pop_right } { t1 }

```

```

__bnvs_seq_if_empty:cTF  __bnvs_seq_if_empty:cTF {<seq core>} {<yes:>} {<no:>}
__bnvs_seq_get_right:ccTF __bnvs_seq_get_right:ccTF {<seq core>} {<t1 core>} {<yes:>} {<no:>}
__bnvs_seq_pop_left:ccTF
__bnvs_seq_pop_right:ccTF

```

```

583 \cs_new:Npn \BNVS_new_conditional_cc:ncnn #1 #2 #3 #4 {
584   \BNVS_new_conditional:cpnn { #1_#2:cc } ##1 ##2 { #4 } {
585     \BNVS_use:ncncn {
586       \BNVS_use_raw:c { #1_#2:NNTF }
587     } { ##1 } { #1 } { ##2 } { #3 } {
588       \prg_return_true:
589     } {
590       \prg_return_false:
591     }
592   }
593 }
594 \BNVS_new_conditional_c:ncn { seq } { if_empty } { T, F, TF }
595 \BNVS_new_conditional_cc:ncnn
596   { seq } { get_right } { t1 } { T, F, TF }
597 \BNVS_new_conditional_cc:ncnn
598   { seq } { pop_left } { t1 } { T, F, TF }
599 \BNVS_new_conditional_cc:ncnn
600   { seq } { pop_right } { t1 } { T, F, TF }

```

6.3.9 Integers

```

__bnvs_int_new:c  __bnvs_int_new:c {<core>}
__bnvs_int_use:c  __bnvs_int_use:c {<core>}
__bnvs_int_zero:c __bnvs_int_incr:c {<core>}
__bnvs_int_inc:c  __bnvs_int_decr:c {<core>}
__bnvs_int_decr:c __bnvs_int_set:cn {<core>} {<value>}
__bnvs_int_set:cn
__bnvs_int_set:cv

```

These are shortcuts to

- `\int_new:c {l__bnvs_<core>_int}`
- `\int_use:c {l__bnvs_<core>_int}`
- `\int_incr:c {l__bnvs_<core>_int}`
- `\int_decr:c {l__bnvs_<core>_int}`
- `\int_set:cn {l__bnvs_<core>_int} {<value>}`

```

601 \BNVS_new_c:nc { int } { new }
602 \BNVS_new_c:nc { int } { use }
603 \BNVS_new_c:nc { int } { zero }
604 \BNVS_new_c:nc { int } { incr }
605 \BNVS_new_c:nc { int } { decr }
606 \BNVS_new_cn:nc { int } { set }
607 \BNVS_new_cv:ncn { int } { set } { int }

```

6.4 Debug facilities

Typesetting file `beanoves.dtx` creates both `beanoves` and `beanoves-debug` style files. The former is intended for everyday use whereas the latter contains supplemental debugging and testing facilities which are intentionally left undocumented. For example, we have aliases for `\group_begin:` and `\group_end:` to allow the display of supplemental informations while debugging. We also have some techniques for coverage as well as inline testing.

6.5 Local variables

We make heavy use of local variables and function scopes. Many functions are executed within a `TEX` group, which ensures no name collision with the caller stack. The number of variables used has not been optimized, nor the `TEX` groups used. Optimization often goes against readability. Next local variables are used by different functions. These are one word letter variables.

`\l__bnvs_A_tl:` ,

`\l__bnvs_Z_tl:` ,

`\l__bnvs_L_tl:` Start, end and lentgh of a range

```

608 \tl_new:N \l__bnvs_A_tl
609 \tl_new:N \l__bnvs_Z_tl
610 \tl_new:N \l__bnvs_L_tl

```

`\l__bnvs_C_tl:` A function name in a call, as regex caught group

`\l__bnvs_I_tl:` A frame identifier

`\l__bnvs_J_tl:` The current frame identifier

`\l__bnvs_S_tl:` A short name, a regex caught group.

`\l__bnvs_R_tl:` A relative, a regex caught group.

`\l__bnvs_P_tl:` A path, a regex caught group.

`\l__bnvs_M_tl:` An assignment modifier, a regex caught group.

```

611 \tl_new:N \l__bnvs_I_tl
612 \tl_new:N \l__bnvs_J_tl
613 \tl_new:N \l__bnvs_K_tl

```

```

614 \tl_new:N \l__bnvs_S_tl
615 \tl_new:N \l__bnvs_R_tl
616 \tl_new:N \l__bnvs_P_tl
617 \tl_new:N \l__bnvs_M_tl

```

`\l__bnvs_D_tl`: To store the scanned material.

```

618 \tl_new:N \l__bnvs_D_tl

```

`\l__bnvs_N_tl`: The representation of an unsigned decimal integer.

```

619 \tl_new:N \l__bnvs_N_tl

```

```

620 \tl_new:N \l__bnvs_B_tl
621 \tl_new:N \l__bnvs_T_tl
622 \tl_new:N \l__bnvs_U_tl
623 \tl_new:N \l__bnvs_V_tl
624 \tl_new:N \l__bnvs_F_tl
625 \tl_new:N \l__bnvs_Q_tl
626 \tl_new:N \l__bnvs_a_tl
627 \tl_new:N \l__bnvs_z_tl
628 \tl_new:N \l__bnvs_l_tl
629 \tl_new:N \l__bnvs_r_tl
630 \tl_new:N \l__bnvs_n_tl
631 \tl_new:N \l__bnvs_ref_tl
632 \tl_new:N \l__bnvs_v_tl
633 \tl_new:N \l__bnvs_b_tl
634 \tl_new:N \l__bnvs_c_tl
635 \tl_new:N \l__bnvs_ans_tl
636 \tl_new:N \l__bnvs_base_tl
637 \tl_new:N \l__bnvsroup_tl
638 \tl_new:N \l__bnvs_scan_tl
639 \tl_new:N \l__bnvs_query_tl
640 \tl_new:N \l__bnvs_n_incr_tl
641 \tl_new:N \l__bnvs_incr_tl
642 \tl_new:N \l__bnvs_plus_tl
643 \tl_new:N \l__bnvs_rhs_tl
644 \tl_new:N \l__bnvs_post_tl
645 \tl_new:N \l__bnvs_suffix_tl
646 \tl_new:N \l__bnvs_index_tl
647 \int_new:N \g__bnvs_call_int
648 \int_new:N \l__bnvs_i_int
649 \seq_new:N \g__bnvs_def_seq
650 \seq_new:N \l__bnvs_a_seq
651 \seq_new:N \l__bnvs_b_seq
652 \seq_new:N \l__bnvs_ans_seq
653 \seq_new:N \l__bnvs_Q_seq
654 \seq_new:N \l__bnvs_R_seq
655 \seq_new:N \l__bnvs_match_seq
656 \seq_new:N \l__bnvs_split_seq
657 \seq_new:N \l__bnvs_P_seq
658 \bool_new:N \l__bnvs_parse_bool
659 \bool_set_false:N \l__bnvs_parse_bool

```

```

660 \bool_new:N \l__bnvs_deep_bool
661 \bool_set_false:N \l__bnvs_deep_bool
662 \bool_new:N \l__bnvs_no_range_bool
663 \bool_set_false:N \l__bnvs_no_range_bool

664 \BNVS_new:cpn { tl_clear_ans: } {
665   \__bnvs_tl_clear:c { ans }
666 }

667 \cs_new:Npn \BNVS_error_ans:x {
668   \__bnvs_tl_put_right:cn { ans } 0
669   \BNVS_error:x
670 }

```

In order to implement the provide feature, we add getters and setters

```

671 \BNVS_new:cpn { set_true:c } #1 {
672   \exp_args:Nc \bool_set_true:N { \BNVS_l:cn { #1 } { bool } }
673 }

674 \BNVS_new:cpn { set_false:c } #1 {
675   \exp_args:Nc \bool_set_false:N { \BNVS_l:cn { #1 } { bool } }
676 }

```

6.6 Infinite loop management

Unending recursivity is managed here.

`\g__bnvs_call_int` Some functions calls, as well as some loop bodies, decrement this counter. When this counter reaches 0, an error is raised or a computation is aborted.

(End of definition for `\g__bnvs_call_int`.)

```

677 \int_const:Nn \c__bnvs_max_call_int { 8192 }

```

`\__bnvs_greset_call:` `\__bnvs_greset_call:`

Reset globally the call stack counter to its maximum value.

```

678 \BNVS_new:cpn { greset_call: } {
679   \int_gset:Nn \g__bnvs_call_int { \c__bnvs_max_call_int }
680 }

```

`\__bnvs_if_call:TF` `\__bnvs_call_do:TF {<yes:>} {<no:>}`

Decrement the `\g__bnvs_call_int` counter globally and execute `<yes:>` if we have not reached 0, `<no:>` otherwise.

```

681 \BNVS_new_conditional:cpnn { if_call: } { T, F, TF } {
682   \int_gdecr:N \g__bnvs_call_int
683   \int_compare:nNnTF \g__bnvs_call_int > 0 {
684     \prg_return_true:
685   } {
686     \prg_return_false:
687   }
688 }

```

6.7 Overlay specification

6.7.1 Registration

We keep track of the $\langle id \rangle! \langle path \rangle$ combinations and provide looping mechanisms. Hereafter, $\langle id \rangle$, $\langle path \rangle$ and $\langle key \rangle$ allways represent pure strings.

```

__bnvs_g_IPK:w  __bnvs_g_IPK:w {<id>} {<path>} {<key>}
__bnvs_g_IP:w   __bnvs_g_IP:w  {<id>} {<path>}
__bnvs_g_I:w    __bnvs_g_I:w   {<id>}

```

Create a unique global name from the arguments.

```

689 \BNVS_new:cpn { g_IPK:w } #1 #2 #3 { g__bnvs@#1!#2/#3_tl }
690 \BNVS_new:cpn { g_IP:w } #1 #2 { g__bnvs@#1!#2_keys_seq }
691 \BNVS_new:cpn { g_I:w } #1 { g__bnvs@#1_paths_seq }

```

$\backslash g_bnvs_I_seq$ Global list of globally registered identifiers.

(End of definition for $\backslash g_bnvs_I_seq$.)

```

692 \seq_new:N \g__bnvs_I_seq

```

6.7.2 Data model

The whole data model is somehow similar to a hierarchical file system: the $\langle id \rangle$ corresponds to the drive, the $\langle path \rangle$ points to a directory, each directory may contain individual files. File named “=” contains initial data whereas file named “@” contains the static resolved data. The latter is initially undefined and becomes empty as soon as the initial data is defined or set. In order to allow index override, files respectively named “ $\langle N \rangle$ ” and “ $@ \langle N \rangle$ ” are used similarly, $\langle N \rangle$ denoting a normalized integer (no leading 0, no leading + sign). The dot (.) is used to separate components of $\langle path \rangle$.

6.7.3 Low level IPK model functions

These are **gset** and **gunset** functions. Each **gset** function must be balanced by a **gunset** function to prevent “memory leaks”.

```

__bnvs_gset:nnnn  __bnvs_gset:nnnn {<id>} {<path>} {<key>} {<def>}
__bnvs_gset:(nnnv|nnne|vnvn|nvnn|nvvn|vvnn|vvvn)

```

Manage the storage. Keep track of all the $\langle path \rangle$ ’s for a given $\langle id \rangle$ and all the $\langle key \rangle$ ’s for a given $\langle id \rangle! \langle path \rangle$ combination.

Implementation details, next macros are defined lazily at the global level:

- $\backslash_bnvs_gset@ \langle id \rangle! \langle path \rangle:nn \{ \langle key \rangle \} \{ \langle def \rangle \}$,
- $\backslash_bnvs_gset@ \langle id \rangle:nnn \{ \langle path \rangle \} \{ \langle key \rangle \} \{ \langle def \rangle \}$,
- $\backslash g_bnvs@ \langle id \rangle!_paths_tl$, records all $\langle path \rangle$ for $\langle id \rangle$,
- $\backslash g_bnvs@ \langle id \rangle! \langle path \rangle_keys_tl$, records all $\langle key \rangle$ for $\langle id \rangle! \langle path \rangle$,
- $\backslash g_bnvs@ \langle id \rangle! \langle path \rangle_keys_tl$,

- `\g__bnvs@<id>!<path>/<key>_tl`, record the `<def>` for the `<id>!<path>/<key>` combination.

```

693 \BNVS_new:cpn { gset:Nn } #1 {
694   \tl_gclear_new:N #1
695   \tl_gset:Nn #1
696 }
697 \BNVS_new:cpn { gset_IPKV:NNw } #1 #2 #3 #4 {
698   \seq_gclear_new:N #1
699   \cs_new:Npn #2 ##1 ##2 {
700     \seq_if_in:NnF #1 { ##1 } {
701       \seq_gput_right:Nn #1 { ##1 }
702     }
703     \exp_args:Nc \__bnvs_gset:Nn {
704       \__bnvs_g_IPK:w { #3 } { #4 } { ##1 }
705     } { ##2 }
706   }
707   #2
708 }
709 \BNVS_new:cpn { gset:NNnnnn } #1 #2 #3 #4 {
710   \cs_if_exist_use:NF #1 {
711     \seq_gput_right:Nn \g__bnvs_I_seq { #3 }
712     \seq_gclear_new:N #2
713     \cs_new:Npn #1 ##1 {
714       \seq_if_in:NnF #2 { ##1 } {
715         \seq_gput_right:Nn #2 { ##1 }
716       }
717       \exp_args:Ncc \__bnvs_gset_IPKV:NNw
718       { \__bnvs_g_IP:w { #3 } { ##1 } }
719       { __bnvs_gset@#3!##1:nn } { #3 } { ##1 }
720     }
721     #1
722   }
723   { #4 }
724 }
725 \BNVS_new:cpn { gset:nnnn } #1 #2 #3 #4 {
726   \cs_if_exist_use:cF { __bnvs_gset@#1!#2:nn } {
727     \exp_args:Ncc \__bnvs_gset:NNnnnn
728     { __bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
729   } { #3 } { #4 }
730 }
731 \BNVS_new:cpn { gset:vnnn } {
732   \BNVS_tl_use:cv { gset:nnnn }
733 }
734 \BNVS_new:cpn { gset:nvnn } #1 {
735   \BNVS_tl_use:nv { \__bnvs_gset:nnnn { #1 } }
736 }
737 \BNVS_new:cpn { gset:vvnn } {
738   \BNVS_tl_use:Nvv \__bnvs_gset:nnnn
739 }
740 \BNVS_new:cpn { gset:nnnv } #1 #2 #3 {
741   \BNVS_tl_use:nv {
742     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
743   }

```

```

744 }
745 \BNVS_new:cpn { gset:vvnv } #1 #2 #3 {
746   \BNVS_tl_use:nv {
747     \__bnvs_gset:vvnn { #1 } { #2 } { #3 }
748   }
749 }
750 \BNVS_new:cpn { gset:nvnv } {
751   \BNVS_tl_use:Nv \__bnvs_gset:nnnv
752 }
753 \cs_generate_variant:Nn \__bnvs_gset:nnnn { nnne }

```

__bnvs_gset:nnv __bnvs_gset:nnv {<id>} {<path>} {<key>}

Syntactic sugar for __bnvs_gset:nnnv {<id>} {<path>} {<key>} {<key>}.

```

754 \BNVS_new:cpn { gset:nnv } #1 #2 #3 {
755   \__bnvs_gset:nnnv { #1 } { #2 } { #3 } { #3 }
756 }

```

__bnvs_gunset:nnn __bnvs_gunset:nnv {<id>} {<path>} {<key>}
__bnvs_gunset:(nvn|vvnn)

Removes the specifications for the <id>{<path>}/{<key>} combination.

```

757 \seq_new:N \l__bnvs_I_seq
758 \BNVS_new:cpn { seq_gremove_once:Nn } #1 #2 {
759   \seq_clear:N \l__bnvs_I_seq
760   \cs_set:Npn \BNVS_seq_gremove_Nn:n ##1 {
761     \tl_if_eq:nnTF { ##1 } { #2 } {
762       \cs_set:Npn \BNVS_seq_gremove_Nn:n #####1 {
763         \seq_put_right:Nn \l__bnvs_I_seq { #####1 }
764       }
765     } {
766       \seq_put_right:Nn \l__bnvs_I_seq { ##1 }
767     }
768   }
769   \seq_map_function:NN #1 \BNVS_seq_gremove_Nn:n
770   \seq_gset_eq:NN #1 \l__bnvs_I_seq
771 }
772 \BNVS_new:cpn { gunset_IP:Nw } #1 #2 #3 {
773   \__bnvs_seq_gremove_once:Nn #1 { #3 }
774   \seq_if_empty:NT #1 {
775     \__bnvs_seq_gremove_once:Nn \g__bnvs_I_seq { #2 }
776     \cs_undefine:c { __bnvs_gset@#2:nnn}
777   }
778 }
779 \BNVS_new:cpn { gunset:NNnnn } #1 #2 #3 #4 #5 {
780   \tl_if_exist:NT #1 {
781     \cs_undefine:N #1
782     \__bnvs_seq_gremove_once:Nn #2 { #5 }
783     \seq_if_empty:NT #2 {
784       \exp_args:Nc \__bnvs_gunset_IP:Nw { \__bnvs_g_I:w { #3 } } { #3 } { #4 }
785       \cs_undefine:c { __bnvs_gset@#3!#4:nn}
786     }
787   }
788 }
789 \BNVS_new:cpn { gunset:nnn } #1 #2 #3 {

```

```

790 \exp_args:Ncc
791 \__bnvs_gunset:NNnnn
792   { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
793   { \__bnvs_g_IP:w { #1 } { #2 } }
794   { #1 } { #2 } { #3 }
795 }
796 \BNVS_new:cpn { gunshot:nvn } #1 {
797   \BNVS_tl_use:nv { \__bnvs_gunset:nnn { #1 } }
798 }
799 \BNVS_new:cpn { gunshot:vvv } {
800   \BNVS_tl_use:Nvv \__bnvs_gunset:nnn
801 }

```

$\backslash_bnvs_is_gset:nnnTF$ $\backslash_bnvs_is_gset:(nvn vvv vxn)TF$	$\backslash_bnvs_is_gset:nnnTF \{ \langle id \rangle \} \{ \langle path \rangle \} \{ \langle key \rangle \} \{ \langle yes: \rangle \} \{ \langle no: \rangle \}$
--	---

Test for the existence of some data for that $\langle id \rangle! \langle path \rangle / \langle key \rangle$ combination.

```

802 \BNVS_new_conditional:cpnn { is_gset:nnn } #1 #2 #3 { T, F, TF } {
803   \tl_if_exist:cTF { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
804   \prg_return_true: \prg_return_false:
805 }
806 \BNVS_new_conditional:cpnn { is_gset:vxn } #1 #2 #3 { T, F, TF } {
807   \exp_args:NNnx \BNVS_tl_use:Nv
808   \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
809   \prg_return_true: \prg_return_false:
810 }
811 \BNVS_new_conditional:cpnn { is_gset:nvn } #1 #2 #3 { T, F, TF } {
812   \BNVS_tl_use:nv { \__bnvs_is_gset:nnnTF { #1 } } { #2 } { #3 }
813   \prg_return_true: \prg_return_false:
814 }
815 \BNVS_new_conditional:cpnn { is_gset:vvv } #1 #2 #3 { T, F, TF } {
816   \BNVS_tl_use:Nvv \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
817   \prg_return_true: \prg_return_false:
818 }

```

6.7.4 Loops

$\backslash_bnvs_foreach_I:N$ $\backslash_bnvs_foreach_I:n$ $\backslash_bnvs_foreach_I_break:$ $\backslash_bnvs_foreach_I_break:n$	$\backslash_bnvs_foreach_I:N \langle function:n \rangle$ $\backslash_bnvs_foreach_I:n \{ \langle :n \text{ code} \rangle \}$ $\backslash_bnvs_foreach_I_break:$ $\backslash_bnvs_foreach_I_break:n \{ \langle code \rangle \}$
---	---

Execute the $\langle function:n \rangle$ or the $\langle :n \text{ code} \rangle$ for each declared identifier. The break commands work like `\seq_map_break:` and

```

819 \BNVS_new:cpn { foreach_I:N } {
820   \seq_map_function:NN \g__bnvs_I_seq
821 }
822 \BNVS_new:cpn { foreach_I:n } {
823   \seq_map_inline:Nn \g__bnvs_I_seq
824 }

```

```

825 \cs_set_eq:NN \__bnvs_foreach_I_break: \seq_map_break:
826 \cs_set_eq:NN \__bnvs_foreach_I_break:n \seq_map_break:n

```

```

\__bnvs_foreach_P:nNTF \__bnvs_foreach_P:nNTF {<id>} <function:nn> {<yes:>} {<no:>}
\__bnvs_foreach_P:nnTF \__bnvs_foreach_P:nnTF {<id>} {<code:nn>} {<yes:>} {<no:>}

```

If $\langle id \rangle$ is a declared identifier, execute $\langle function:nn \rangle$ or $\langle code:nn \rangle$ for each combination of $\langle id \rangle$ and its associate $\langle path \rangle$ s. Both are the arguments passed to $\langle function:nn \rangle$ or $\langle :nn code \rangle$ body (through ##1 and ##2).

```

827 \BNVS_new_conditional:cpnn { foreach_P:nN } #1 #2 { T, F, TF } {
828   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
829     \seq_map_inline:cn {
830       \__bnvs_g_I:w { #1 }
831     } {
832       #2 { #1 } { ##1 }
833     }
834     \prg_return_true:
835   } {
836     \prg_return_false:
837   }
838 }

839 \BNVS_new_conditional:cpnn { foreach_P:nn } #1 #2 { T, F, TF } {
840   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
841     \cs_set:Npn \BNVS_foreach_P_nn:nn ##1 ##2 { #2 }
842     \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
843       \BNVS_foreach_P_nn:nn { #1 } { ##1 }
844     }
845     \prg_return_true:
846   } {
847     \prg_return_false:
848   }
849 }

```

```

\__bnvs_foreach_K:nnN \__bnvs_foreach_K:nnN {<id>} {<path>} <function:nnn>
\__bnvs_foreach_K:nnn \__bnvs_foreach_K:nnn {<id>} {<path>} {<code:nnn>}

```

Execute the $\langle function:nnn \rangle$ or the $\langle code:nnn \rangle$ body for each of $\langle id \rangle!$ $\langle tag \rangle/$ $\langle key \rangle$ combination. These are the arguments to the function or code body.

```

850 \BNVS_new:cpn { foreach_K:NnnN } #1 #2 #3 #4 {
851   \seq_if_exist:NT #1 {
852     \seq_map_inline:Nn #1 {
853       #4 { #2 } { #3 } { ##1 }
854     }
855   }
856 }

857 \BNVS_new:cpn { foreach_K:nnN } #1 #2 #3 {
858   \exp_args:Nc \__bnvs_foreach_K:NnnN
859   { \__bnvs_g_IP:w { #1 } { #2 } }
860   { #1 } { #2 } #3
861 }

```

```

862 \BNVS_new:cpn { foreach_K:nnn } #1 #2 #3 {
863   \cs_set:Npn \BNVS_foreach_K_nnn:w ##1 ##2 ##3 { #3 }
864   \__bnvs_foreach_K:nnN { #1 } { #2 } \BNVS_foreach_K_nnn:w
865 }

```

```

\__bnvs_foreach_R:nnNTF \__bnvs_foreach_R:nnNTF {<id>} {<tag>} {<function:nn> {<yes:>} {<no:>}}
\__bnvs_foreach_R:nnnTF \__bnvs_foreach_R:nnnTF {<id>} {<tag>} {<code:nn>} {<yes:>} {<no:>}}

```

If the $\langle id \rangle$ is known, execute the $\langle function:nn \rangle$ or the $\langle code:nn \rangle$ body for each of $\langle id \rangle! \langle tag \rangle. \langle subtag \rangle$ combination. The $\langle id \rangle$ and $\langle tag \rangle. \langle subtag \rangle$ are the arguments of the function and the function body. After that the $\langle yes: \rangle$ code is executed if some combination is found, otherwise it executes $\langle no: \rangle$ code.

```

866 \BNVS_new_conditional:cpnn { foreach_R:nnn } #1 #2 #3 { T, F, TF } {
867   \cs_set:Npn \BNVS_foreach_R_nnnTF:nn ##1 ##2 {
868     #3
869   }
870   \__bnvs_foreach_R:nnNTF { #1 } { #2 } \BNVS_foreach_R_nnnTF:nn
871   \prg_return_true: \prg_return_false:
872 }

873 \BNVS_new_conditional:cpnn { foreach_R:nnN } #1 #2 #3 { T, F, TF } {
874   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {

```

This is not reentrant.

```

875   \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
876     \cs_set:Npn \BNVS_foreach_R_nnnTF_return: { \prg_return_true: }
877     \str_if_in:nnTF { ~ ##1 . } { ~ #2 . } {
878       #3 { #1 } { ##1 }
879     } {
880     }
881   }
882   \BNVS_foreach_R_nnnTF_return:
883 } {
884   \prg_return_false:
885 }
886 }

```

```

\__bnvs_foreach_IP:N \__bnvs_foreach_IP:N {<function:nn>
\__bnvs_foreach_IP:n \__bnvs_foreach_IP:n {<code:nn>}

```

Execute the $\langle function:nn \rangle$ or the $\langle code:nn \rangle$ for each $\langle id \rangle! \langle path \rangle$ combination.

```

887 \BNVS_new:cpn { foreach_IP:N } #1 {
888   \__bnvs_foreach_I:n {
889     \__bnvs_foreach_P:nNT { ##1 } #1 { }
890   }
891 }

892 \BNVS_new:cpn { foreach_IP:NT } #1 #2 {
893   \__bnvs_foreach_I:n {
894     \__bnvs_foreach_P:nNT { ##1 } #1 { #2 }
895   }
896 }

```

```

897 \BNVS_new:cpn { foreach_IP:n } #1 {
898   \cs_set:Npn \BNVS_foreach_IP_n:nn ##1 ##2 { #1 }
899   \__bnvs_foreach_I:n {
900     \__bnvs_foreach_P:nNT { ##1 } \BNVS_foreach_IP_n:nn { }
901   }
902 }

```

6.7.5 Path function

__bnvs_walk_path:nnnn __bnvs_walk_path:nnnn {<path>} {<relative>} {<first:n>} {<other:n>}

<path> is a dot separated list of components.

```

903 \BNVS_seq_new:c { walk }
904 \BNVS_set:cpn { walk_path:nnnn } #1 #2 #3 #4 {
905   \BNVS_set:cpn { walk_path_ii:n } ##1 { #2 }
906   \BNVS_set:cpn { walk_path_iii:n } ##1 { #3 }
907   \BNVS_set:cpn { walk_path_iiii:n } ##1 { #4 }
908   \BNVS_seq_use:Nc \seq_set_split:Nnn { walk } . { #1 }
909   \BNVS_set:cpn { walk_path_next: } { }
910   \cs_set:Npn \BNVS:n ##1 {
911     \BNVS_set:cpn { walk_path_next: } { }
912     \__bnvs_walk_path_ii:n { ##1 }
913   }
914   \cs_set:Npn \BNVS:n #####1 {
915     \__bnvs_walk_path_iii:n { ##1 }
916     \BNVS_set:cpn { walk_path_next: } { }
917     \cs_set:Npn \BNVS:n #####1 {
918       \__bnvs_walk_path_iiii:n { #####1 }
919     }
920     \__bnvs_walk_path_iiii:n { #####1 }
921   }
922 }
923 \BNVS_seq_use:Nc \seq_map_function:NN { walk } \BNVS:n
924 \__bnvs_walk_path_next:
925 }

```

__bnvs_foreach_relative:nn __bnvs_foreach_relative:nn {<relative>} {<code:n>}

<relative> is a dot separated list of components starting with a dot.

```

926 \BNVS_set:cpn { foreach_relative:nn } #1 #2 {
927   \BNVS_set:cpn { foreach_relative:n } ##1 { #2 }
928   \BNVS_seq_use:Nc \seq_set_split:Nnn { walk } . { #1 }
929   \cs_set:Npn \BNVS:n ##1 {
930     \cs_set:Npn \BNVS:n { \__bnvs_foreach_relative:n }
931   }
932   \BNVS_seq_use:Nc \seq_map_function:NN { walk } \BNVS:n
933 }

```

6.7.6 Low level IP model functions

6.7.7 Low level I model functions

6.7.8 Getting

<u>_bnvs_if_get:nnnTF</u>	<u>_bnvs_if_get:nnnTF</u> {<id> } {<path>} {<key>} {<yes:n>} {<no:>}
<u>_bnvs_if_spec:nnnTF</u>	<u>_bnvs_if_spec:nnnTF</u> {<id> } {<path>} {<key>} {<yes:n>} {<no:>}

The _bnvs_if_get:nnnTF variant calls <yes:n> with what was stored for <id>!<path>/<key>. Otherwise executes <no:> without changing the contents of the <ans> t1 variable.

The _spec: variant is similar except that it uses <id>!<path>/<key> combinations when available or !<path>/<key> otherwise.

```

934 \BNVS_new:cpn { if_get:nnnTF } #1 #2 #3 #4 #5 {
935   \\_bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
936     \cs_set:Npn \BNVS:n ##1 { #4 }
937     \exp_args:Nv \BNVS:n { \\_bnvs_g_IPK:w { #1 } { #2 } { #3 } }
938   } {
939     #5
940   }
941 }

942 \BNVS_new:cpn { if_spec:nnnTF } #1 #2 #3 #4 #5 {
943   \\_bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
944     \cs_set:Npn \BNVS:n ##1 { #4 }
945     \exp_args:Nv \BNVS:n { \\_bnvs_g_IPK:w { #1 } { #2 } { #3 } }
946   } {
947     \tl_if_empty:nTF { #1 } {
948       #5
949     } {
950       \\_bnvs_is_gset:nnnTF {} { #2 } { #3 } {
951         \cs_set:Npn \BNVS:n ##1 { #4 }
952         \exp_args:Nv \BNVS:n { \\_bnvs_g_IPK:w {} { #2 } { #3 } }
953       } {
954         #5
955       }
956     }
957   }
958 }
```

<u>_bnvs_if_get:nnncTF</u>	<u>_bnvs_if_get:nnncTF</u> {<id> } {<path>} {<key>} {<ans>} {<yes:>} {<no:>}
<u>_bnvs_if_spec:nnncTF</u>	<u>_bnvs_if_spec:nnncTF</u> {<id> } {<path>} {<key>} {<ans>} {<yes:>} {<no:>}
<u>_bnvs_if_get:nncTF</u>	<u>_bnvs_if_get:nncTF</u> {<id> } {<path>} {<ans>} {<yes:>} {<no:>}
<u>_bnvs_if_get:vcTF</u>	

The _bnvs_if_get:nnnc... variant puts what was stored for <id>!<path>/<key> into the <ans> variable, if any, then executes <yes:>. Otherwise executes <no:> without changing the contents of the <ans> t1 variable.

_bnvs_if_get:nncTF is a convenient shortcut for _bnvs_if_get:nnncTF when <key> and <ans> happen to be the same.

The _spec: variant is similar except that it uses <id>!<path>/<key> combinations when available or !<path>/<key> otherwise.

```

959 \BNVS_new_conditional:cpnn { if_get:nnnc } #1 #2 #3 #4 { T, F, TF } {
```

```

960  \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
961    \BNVS_tl_use:Nc \cs_set_eq:Nc
962      { #4 } { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
963    \prg_return_true:
964  } {
965    \prg_return_false:
966  }
967 }
968 \BNVS_undefine:c { if_get:nnnc }

969 \BNVS_new_conditional:cpnn { if_spec:nnnc } #1 #2 #3 #4 { T, F, TF } {
970   \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
971     \tl_set_eq:cc { \BNVS_l:cn { #4 } { tl } } {
972       \__bnvs_g_IPK:w { #1 } { #2 } { #3 }
973     }
974     \prg_return_true:
975   } {
976     \tl_if_empty:nTF { #1 } {
977       \prg_return_false:
978     } {
979       \__bnvs_if_spec:nnncTF {} { #2 } { #3 } { #4 }
980       \prg_return_true: \prg_return_false:
981     }
982   }
983 }
984 \BNVS_undefine:c { if_spec:nnnc }

985 \BNVS_new_conditional:cpnn { if_get:nnc } #1 #2 #3 { T, F, TF } {
986   \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #3 }
987   \prg_return_true: \prg_return_false:
988 }

989 \BNVS_new_conditional:cpnn { if_get:vvc } #1 #2 #3 { T, F, TF } {
990   \BNVS_tl_use:Nvv \__bnvs_if_get:nncTF { #1 } { #2 } { #3 }
991   \prg_return_true: \prg_return_false:
992 }

```

__bnvs_if_resolved:nncTF __bnvs_if_resolved:nnncTF {<id>} {<path>} {<ans>} {<yes:>} {<no:>}

Execute <yes:> if resolution succeeded for either <id>!<<path>/r or the fallback !<path>/r, otherwise execute <no:>.

```

993 \BNVS_new_conditional:cpnn { if_resolved:nnc } #1 #2 #3 { T, F, TF } {
994   \__bnvs_if_get:nnncTF { #1 } { #2 } r { #3 } {
995     \prg_return_true:
996     \prg_return_false:
997   } {
998     \tl_if_empty:nTF { #1 } {
999       \prg_return_false:
1000     } {
1001       \__bnvs_if_resolved:nnncTF {} { #2 } r { #3 }
1002       \prg_return_true:
1003       \prg_return_false:
1004     }
1005   }

```

```

1006 }
1007 \BNVS_undefine:c { if_resolved:nnc }

```

```

\__bnvs_if_resolved:nnncTF \__bnvs_if_resolved:nnncTF {<id>} {<path>} {<key>} {<ans>} {<yes:>} {<no:>}

```

Execute $\langle\text{yes:}\rangle$ if resolution succeeded for either $\langle\text{id}\rangle!\langle\text{path}\rangle/\langle\text{key}\rangle$ or the fallback $\langle\text{path}\rangle/\langle\text{key}\rangle$, otherwise execute $\langle\text{no:}\rangle$.

```

1008 \BNVS_new_conditional:cpnn { if_resolved:nnnc } #1 #2 #3 #4 { T, F, TF } {
1009   \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #4 } {
1010     \__bnvs_is_will_gset:cTF { #3 }
1011     \prg_return_true: \prg_return_false:
1012   } {
1013     \tl_if_empty:nTF { #1 } {
1014       \prg_return_false:
1015     } {
1016       \__bnvs_if_resolved:nnncTF { #1 } { #2 } { #3 } { #4 }
1017       \prg_return_true: \prg_return_false:
1018     }
1019   }
1020 }

```

```

\__bnvs_if_resolved_IP:wTF \__bnvs_if_resolved_IP:wTF {<id>} {<path>} {<yes:n>} {<no:>}

```

If $\langle\text{id}\rangle!\langle\text{path}\rangle/\text{r}$ has been set, execute $\langle\text{yes:n}\rangle$ code with that value as argument, otherwise execute $\langle\text{no:}\rangle$ code.

```

1021 \BNVS_tl_new:c { if_resolved_IP:wTF }
1022 \BNVS_new:cpn { if_resolved_IP:wTF } #1 #2 {
1023   \__bnvs_if_get:nnncTF { #1 } { #2 } r { if_resolved_IP:wTF } {
1024     \BNVS_tl_use:Nv \BNVS_apply_i:nnn { if_resolved_IP:wTF }
1025   } {
1026     \use_ii:nn
1027   }
1028 }

```

```

\__bnvs_require:nnnc \__bnvs_require:nnnc {<id>} {<path>} {<key>} {<ans>}

```

```

\__bnvs_require:nnc \__bnvs_require:nnc {<id>} {<path>} {<ans>}

```

```

\__bnvs_require:vvc

```

Calls `\__bnvs_if_get:nnncTF`, does nothing on success and, on failure, does nothing or raise when in debug mode. `\__bnvs_require:nnc` is a shortcut for the more qualified function `\__bnvs_require:nnnc` when $\langle\text{key}\rangle$ and $\langle\text{ans}\rangle$ happen to be identical.

```

1029 \BNVS_new:cpn { require:nnnc } #1 #2 #3 #4 {
1030   \__bnvs_if_get:nnncF { #1 } { #2 } { #3 } { #4 } {
1031     \BNVS_fatal_unreachable:
1032   }
1033 }

```

```

\__bnvs_gunset:nn \__bnvs_gunset:nn {<id>} {<path>}

```

```

\__bnvs_gunset:(nv|vn|vv)

```

Removes the specifications for the $\langle\text{id}\rangle!\langle\text{path}\rangle$ and all possible keys combination.

```

1034 \BNVS_new:cpn { gunset:NNNnn } #1 #2 #3 #4 #5 {
1035   \seq_map_inline:Nn #1 {
1036     \__bnvs_gunset:nnn { #4 } { #5 } { ##1 }
1037   }
1038   \cs_undefine:N #1
1039   \__bnvs_seq_gremove_once:Nn #2 { #5 }
1040   \seq_if_empty:NT #2 {
1041     \cs_undefine:N #2
1042     \__bnvs_seq_gremove_once:Nn #3 { #4 }
1043     \cs_undefine:c { \__bnvs_gset@ #4 :nnn }
1044   }
1045 }
1046 \BNVS_new:cpn { gunset:Nnn } #1 #2 #3 {
1047   \seq_map_inline:Nn #1 {
1048     \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
1049       \exp_args:Nc \__bnvs_gunset:NNNnn
1050         { \__bnvs_g_IP:w { #2 } { ##1 } }
1051       #1 \g__bnvs_I_seq { #2 } { ##1 }
1052       \cs_undefine:c { \__bnvs_gset@{ #2 } { ##1 }:nn }
1053     }
1054   }
1055 }
1056 \BNVS_new:cpn { gunset:nn } #1 #2 {
1057   \exp_args:Nc
1058   \__bnvs_gunset:Nnn { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
1059 }
1060 \BNVS_new:cpn { gunset:nv } #1 {
1061   \BNVS_tl_use:nv { \__bnvs_gunset:nn { #1 } }
1062 }
1063 \BNVS_new:cpn { gunset:vn } {
1064   \BNVS_tl_use:cv { gunset:nn }
1065 }
1066 \BNVS_new:cpn { gunset:vv } {
1067   \BNVS_tl_use:Nvv \__bnvs_gunset:nn
1068 }

```

`\__bnvs_gunset_deep:nn`
`\__bnvs_gunset_deep:(nv|vv)`

`\__bnvs_gunset_deep:nn` {<id>} {<path>}

Removes the specifications for each existing <id>!
<path>.<subpath> combination.

```

1069 \BNVS_new:cpn { gunset_deep:Nnn } #1 #2 #3 {
1070   \seq_if_exist:NT #1 {
1071     \seq_map_inline:Nn #1 {
1072       #3 is exactly ##1 or start with ##1., which is equivalent to #3. starts with ##1.:
1073       \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
1074         \__bnvs_gunset:nn { #2 } { ##1 }
1075       }
1076     }
1077   }

```

```

1078 \BNVS_new:cpn { gunset_deep:nn } #1 #2 {
1079   \exp_args:Nc \__bnvs_gunset_deep:Nnn
1080   { \__bnvs_g_I:w { #1 } }
1081   { #1 } { #2 }
1082 }

1083 \BNVS_new:cpn { gunset_deep:nv } #1 {
1084   \BNVS_tl_use:nv { \__bnvs_gunset_deep:nn { #1 } }
1085 }

1086 \BNVS_new:cpn { gunset_deep:vv } {
1087   \BNVS_tl_use:Nvv \__bnvs_gunset_deep:nn
1088 }

```

```

\__bnvs_gunset:n \__bnvs_gunset:n {<id>}
\__bnvs_gunset:  \__bnvs_gunset:

```

Removes the specifications for the $\langle id \rangle$ and all possible tag combination. The second does the same for all possible $\langle id \rangle$.

```

1089 \BNVS_new:cpn { gunset:NNNn } #1 #2 #3 #4 {
1090   \cs_if_exist:NT #1 {
1091     \cs_undefine:N #1
1092     \seq_map_inline:Nn #2 {
1093       \__bnvs_gunset:nn { #4 } { ##1 }
1094     }
1095     \cs_undefine:N #2
1096     \__bnvs_seq_gremove_once:Nn #3 { #4 }
1097   }
1098 }

1099 \BNVS_new:cpn { gunset:n } #1 {
1100   \exp_args:Ncc \__bnvs_gunset:NNNn
1101   { \__bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } }
1102   \g__bnvs_I_seq { #1 }
1103 }

1104 \BNVS_new:cpn { gunset: } {
1105   \seq_map_inline:Nn \g__bnvs_I_seq {
1106     \__bnvs_gunset:n { ##1 }
1107   }
1108 }

```

```

\__bnvs_VS_gunset: \__bnvs_VS_gunset:

```

Removes any value for the “V” and “S” keys and any $\langle id \rangle!$ $\langle path \rangle$ combination.

```

1109 \BNVS_new:cpn { VS_gunset: } {
1110   \__bnvs_foreach_IP:n {
1111     \__bnvs_gunset:nnn { ##1 } { ##2 } V
1112     \__bnvs_gunset:nnn { ##1 } { ##2 } S
1113   }
1114 }

```

6.7.9 Circular dependencies

`\_bnvs_will_gset:nnn` `\_bnvs_will_gset:nnn {⟨id⟩} {⟨path⟩} {⟨key⟩}`

To manage circular dependencies. After this command, `\_bnvs_is_will_gset:nnnTF` is true for the same arguments.

```

1115 \BNVS_new:cpn { will_gset:nnn } #1 #2 #3 {
1116   \_bnvs_gset:nnnn { #1 } { #2 } { #3 } { \q_no_value }
1117 }
```

`\_bnvs_is_will_gset:cTF` `\_bnvs_is_will_gset:cTF {⟨in⟩} {⟨yes:⟩} {⟨no:⟩}`

To manage circular dependencies. See `\_bnvs_will_gset:nnn` above.

```

1  \_bnvs_will_gset:nnn {⟨id⟩} {⟨path⟩} {⟨key⟩}
2  ...
3  \_bnvs_if_get:nnncT {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨in⟩} {
4    ...
5    \_bnvs_is_will_gset:cTF
6      {⟨in⟩}
7      {⟨yes:⟩}
8      {⟨no:⟩}
9    ...
10 }
```

```

1118 \BNVS_new_conditional:cpnn { is_will_gset:c } #1 { T, F, TF } {
1119   \BNVS_tl_use:Nv \quark_if_no_value:nTF { #1 } {
1120     \prg_return_true:
1121   } {
1122     \prg_return_false:
1123   }
1124 }
```

6.7.10 Spring cleaning

`\_bnvs_gclear:nn` `\_bnvs_gclear:nn {⟨id⟩} {⟨path⟩}`
`\_bnvs_gclear:nv` `\_bnvs_gclear:n {⟨id⟩}`
`\_bnvs_gclear:` `\_bnvs_gclear:`
`\_bnvs_greset:nn` `\_bnvs_greset:nn {⟨id⟩} {⟨path⟩}`
`\_bnvs_greset:nv` `\_bnvs_greset:n {⟨id⟩}`
`\_bnvs_greset:` `\_bnvs_greset:`

The first ones clear out every data stored for `⟨id⟩!⟨path⟩`, including deeper and registration data. When `⟨path⟩` is omitted, any known path is considered. When `⟨id⟩` is omitted, any known id is considered. The second one only clears out the data created on the fly after the initialization step. The initial data (for key =) is left untouched.

```

1125 \BNVS_new:cpn { greset:nn } #1 #2 {
```

```

1126 \__bnvs_foreach_P:nnT { #1 } {
1127   \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
1128     \__bnvs_foreach_K:nnn { #1 } { ##2 } {
1129       \tl_if_in:nnF { \q_bnvs ####3 } { \q_bnvs = } {
1130         \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1131       }
1132     }
1133   }
1134 } { }
1135 }

1136 \BNVS_new:cpn { greset:nv } #1 {
1137   \BNVS_tl_use:nv { \__bnvs_greset:nn { #1 } }
1138 }

```

Clears out every data associated to $\langle id \rangle!$ $\langle path \rangle$, and any $\langle id \rangle!$ $\langle path \rangle.$ $\langle subpath \rangle$.

```

1139 \BNVS_new:cpn { gclear:nn } #1 #2 {
1140   \__bnvs_foreach_P:nnT { #1 } {
1141     \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
1142       \__bnvs_foreach_K:nnn { #1 } { ##2 } {
1143         \tl_if_in:nnTF { \q_bnvs ####3 } { \q_bnvs = } {
1144           \__bnvs_gset:nnnn { #1 } { ##2 } { ####3 } { 0 }
1145         } {
1146           \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1147         }
1148       }
1149     }
1150   } { }
1151 }

1152 \BNVS_new:cpn { gclear:n } #1 {
1153   \__bnvs_foreach_P:nNT { #1 } \__bnvs_gclear:nn {}
1154 }

1155 \BNVS_new:cpn { gclear: } {
1156   \__bnvs_foreach_IP:N \__bnvs_gclear:nn
1157 }

1158 \BNVS_new:cpn { greset:n } #1 {
1159   \__bnvs_foreach_P:nNT { #1 } \__bnvs_greset:nn {}
1160 }

1161 \BNVS_new:cpn { greset: } {
1162   \__bnvs_foreach_IP:N \__bnvs_greset:nn
1163 }

```

__bnvs_gclear_resolved:nn __bnvs_gclear_resolved:nn { $\langle id \rangle$ } { $\langle path \rangle$ }

__bnvs_gclear_resolved:nv

Clear the resolved data, aka any key that does not start with “=”.

```

1164 \quark_new:N \q__bnvs_X

```

```

1165 \BNVS_new:cpn { gclear_resolved:nn } #1 #2 {
1166   \__bnvs_foreach_IP:nnn { #1 } { #2 } {
1167     \tl_if_in:nnF { \q__bnvs_X ##1 } { \q__bnvs_X = } {
1168       \__bnvs_gunset:nnn { #1 } { #2 } { ##1 }
1169     }
1170   }
1171 }

1172 \BNVS_new:cpn { gclear_resolved:nv } #1 {
1173   \BNVS_tl_use:nv {
1174     \__bnvs_gclear_resolved:nn { #1 }
1175   }
1176 }

```

6.7.11 The provide mode

`\l__bnvs_provide_bool` Manage the provide mode.

```

1177 \bool_new:N \l__bnvs_provide_bool

(End of definition for \l__bnvs_provide_bool.)

1178 \BNVS_new:cpn { provide_ON: } {
1179   \__bnvs_set_true:c { provide }
1180 }

1181 \BNVS_new:cpn { provide_OFF: } {
1182   \__bnvs_set_false:c { provide }
1183 }
1184 \__bnvs_provide_OFF:

```

<code>__bnvs_is_gset_and_provide:nnn</code>	<code><i>TF</i></code>	<code>__bnvs_is_gset_and_provide:nnn</code>	<code><i>TF</i></code>	<code>{⟨id⟩}</code>	<code>{⟨path⟩}</code>	<code>{⟨key⟩}</code>
<code>__bnvs_is_gset_and_provide:(nvn vvn)</code>	<code><i>TF</i></code>	<code>{⟨yes:⟩}</code>	<code>{⟨no:⟩}</code>			
<code>__bnvs_is_gset_and_provide:nn</code>	<code><i>TF</i></code>	<code>__bnvs_is_gset_and_provide:nn</code>	<code><i>TF</i></code>	<code>{⟨id⟩}</code>	<code>{⟨path⟩}</code>	<code>{⟨yes:⟩}</code>
				<code>{⟨no:⟩}</code>		

Execute `⟨yes:⟩` when in provide mode and `⟨id⟩!⟨path⟩/⟨key⟩` is known, `⟨no:⟩` otherwise.
When not specified, `⟨key⟩` is “=”.

```

1185 \BNVS_new_conditional:cpnn { is_gset_and_provide:nnn }
1186   #1 #2 #3 { T, F, TF } {
1187     \__bnvs_if:cTF { provide } {
1188       \if_cs_exist:w \__bnvs_g_IPK:w { #1 } { #2 } { #3 } \cs_end:
1189       \prg_return_true:
1190     }
1191     \else:
1192       \prg_return_false:
1193     \fi:
1194   } {
1195     \prg_return_false:
1196   }

```

```

1197 \BNVS_new_conditional:cpnn { is_gset_and_provide:nvn }
1198         #1 #2 #3 { T, F, TF } {
1199     \BNVS_tl_use:nv {
1200         \__bnvs_is_gset_and_provide:nnnTF { #1 }
1201     } { #2 } { #3 }
1202     \prg_return_true: \prg_return_false:
1203 }

1204 \BNVS_new_conditional:cpnn { is_gset_and_provide:vvv }
1205         #1 #2 #3 { T, F, TF } {
1206     \BNVS_tl_use:Nvv \__bnvs_is_gset_and_provide:nnnTF { #1 } { #2 } { #3 }
1207     \prg_return_true: \prg_return_false:
1208 }

```

```

\__bnvs_gprovide:TnnnnF \__bnvs_gprovide:TnnnnF {<pre code>} {<id>} {<path>} {<key>} {<value>} {<no:>}

```

```

\__bnvs_gprovide:TvnnnF Execute <no:> exclusively when not in provide mode. Does nothing when something was
\__bnvs_gprovide:TvvnnF set for the <id>!\<path>/\<key> combination. Execute <pre code> before setting.

```

```

1209 \BNVS_new:cpn { gprovide:TnnnnF } #1 #2 #3 #4 #5 {
1210     \__bnvs_if:cTF { provide } {
1211         \__bnvs_is_gset:nnnF { #2 } { #3 } { #4 } {
1212             #1
1213             \__bnvs_gset:nnnn { #2 } { #3 } { #4 } { #5 }
1214         }
1215     }
1216 }

1217 \BNVS_new:cpn { gprovide:TvnnnF } #1 {
1218     \BNVS_tl_use:nv { \__bnvs_gprovide:TnnnnF { #1 } }
1219 }
1220 \BNVS_new:cpn { gprovide:TvvnnF } #1 {
1221     \BNVS_tl_use:nvv { \__bnvs_gprovide:TnnnnF { #1 } }
1222 }

```

```

\__bnvs_is_provided:nnnTF \__bnvs_is_provided:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}

```

If <id>!\<path>/\<key> has been successfully resolved and not unset since then, execute <yes:>. Otherwise execute <no:>.

```

1223 \tl_new:N \l__bnvs_is_provided_tl
1224 \BNVS_new_conditional:cpnn { is_provided:nnn } #1 #2 #3 { T, F, TF } {
1225     \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { is_provided } {
1226         \__bnvs_is_will_gset:cTF { is_provided } {
1227             \prg_return_false:
1228         } {
1229             \prg_return_true:
1230         }
1231     } {
1232         \prg_return_false:
1233     }
1234 }

```

6.7.12 Initialize

`\\_bnvs_ginit:nnnn` `\\_bnvs_ginit:nnnn {<id>} {<short>} {<relative>} {<definition>}`

Called by `\\_bnvs_gdef:nnnn`. This function supports provide mode by calling `\\_bnvs_is_gset_and_provide:nnnTF`. Somehow a shortcut to `\\_bnvs_gset:nnnn` with key “=”. `<relative>` is a list of components, if it is not empty, each intermediate path is modified or defined appropriately by calling `\\_bnvs_greset:nn` among other things.

```

1235 \BNVS_new:cpn { ginit:nnnn } #1 #2 #3 #4 {
1236   \tl_if_empty:nTF { #3 } {
1237     \\_bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
1238       \\_bnvs_gunset_deep:nn { #1 } { #2 }
1239       \\_bnvs_gset:nnnn { #1 } { #2 } = { #4 }
1240     }
1241   } {
1242     \BNVS_begin:
1243     \\_bnvs_greset:nn { #1 } { #2 }
1244     \\_bnvs_is_gset:nnnF { #1 } { #2 } = {
1245       \\_bnvs_gset:nnnn { #1 } { #2 } = {}
1246     }
1247     \\_bnvs_tl_set:cn P { #2 }
1248     \\_bnvs_foreach_relative:nn { #3 } {
1249       \\_bnvs_tl_put_right:cn P { . ##1 }
1250       \\_bnvs_greset:nv { #1 } P
1251       \\_bnvs_is_gset:nvnF { #1 } P = {

```

Each intermediate step is just defined empty when not already defined. The last step setting is overridden just after the loop.

```

1252       \\_bnvs_gset:nvnn { #1 } P = {}
1253     }
1254   }
1255   \\_bnvs_is_gset_and_provide:nvnF { #1 } P = {
1256     \\_bnvs_gunset_deep:nv { #1 } P
1257     \\_bnvs_gset:nvnn { #1 } P = { #4 }
1258   }
1259   \BNVS_end:
1260 }
1261 }

```

`\\_bnvs_ginit:nnn` `\\_bnvs_ginit:nnn {<id>} {<path>} {<simple definition>}`
`\\_bnvs_ginit:(nnv|vvn)`

Called by `\\_bnvs_gdef:nnn`. Somehow a shortcut to `\\_bnvs_gset:nnnn` with key “=”. If `<path>` has more than one component, each intermediate path is modified or defined appropriately. This function supports provide mode by calling `\\_bnvs_is_gset_and_provide:nnnTF`.

```

1262 \BNVS_new:cpn { ginit:nnn } #1 #2 {
1263   \\_bnvs_ginit:nnnn { #1 } { #2 } {}
1264 }

1265 \BNVS_new:cpn { ginit:nnv } #1 #2 {
1266   \BNVS_tl_use:nv { \\_bnvs_ginit:nnn { #1 } { #2 } }
1267 }

```

```

1268 \BNVS_new:cpn { ginit:vvv } {
1269   \BNVS_tl_use:Nvv \__bnvs_ginit:nnn
1270 }

```

__bnvs_if_should_ginit:nnTF __bnvs_if_should_ginit:nnTF {<id>} {<path>} {<yes:>} {<no:>}

Execute <yes:> when in provide mode and <id>{<tag>} combination is not initialized by __bnvs_ginit:nnn. Execute <no:> otherwise.

```

1271 \BNVS_new_conditional:cpnn { if_should_ginit:nn } #1 #2 { T, F, TF } {
1272   \__bnvs_if:cTF { provide } {
1273     \__bnvs_is_ginit:nnTF { #1 } { #2 } {
1274       \prg_return_false:
1275     } {
1276       \prg_return_true:
1277     }
1278   } {
1279     \prg_return_true:
1280   }
1281 }
1282 \BNVS_undefine:c { if_should_ginit:nn }

```

__bnvs_is_ginit:nnTF __bnvs_is_ginit:nnTF {<id>} {<path>} {<yes:>} {<no:>}

Test for the existence of some data for that <id>{<path>} combination.

```

1283 \BNVS_new_conditional:cpnn { is_ginit:nn } #1 #2 { T, F, TF } {
1284   \__bnvs_is_gset:nnnTF { #1 } { #2 } = \prg_return_true: \prg_return_false:
1285 }
1286 \BNVS_undefine:c { is_ginit:nn }

```

__bnvs_if_ginit:nncTF __bnvs_if_ginit:nncTF {<id>} {<tag>} {<var>} {<yes:>} {<no:>}

Test for the existence of some initialization data for that <id>{<path>} combination. On success, <var> is set to this data.

```

1287 \BNVS_new_conditional:cpnn { if_ginit:nnc } #1 #2 #3 { T, F, TF } {
1288   \__bnvs_if_get:nncTF { #1 } { #2 } = { #3 } \prg_return_true: \prg_return_false:
1289 }
1290 \BNVS_undefine:c { if_ginit:nnc }

```

6.8 Regular expressions

\c__bnvs_index_regex Signed integer.

(End of definition for \c__bnvs_index_regex.)

```

1291 \regex_const:Nn \c__bnvs_index_regex {
1292   [-+\s]* \d+
1293 }

```

\c__bnvs_dot_index_regex Signed integer with a dot . just before.

(End of definition for \c__bnvs_dot_index_regex.)

```

1294 \regex_const:Nn \c__bnvs_dot_index_regex {
1295   \. \ur{c__bnvs_index_regex}
1296 }

```

`\c__bnvs_A_dot_index_Z_regex` Full signed integer with a dot . jus before.

(End of definition for \c__bnvs_A_dot_index_Z_regex.)

```

1297 \regex_const:Nn \c__bnvs_A_dot_index_Z_regex {
1298   \A \ur { c__bnvs_dot_index_regex } \Z
1299 }

```

`\c__bnvs_A_index_Z_regex` Signed integer with no capture groups.

(End of definition for \c__bnvs_A_index_Z_regex.)

```

1300 \regex_const:Nn \c__bnvs_A_index_Z_regex { \A \ur{c__bnvs_index_regex} \Z }

```

`\c__bnvs_S_regex` This regular expression is used for short names. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit. It must contain one uppercase letter.

```

1301 \regex_const:Nn \c__bnvs_S_regex {
1302   (?:[a-z_][a-z0-9_]*)?[A-Z][[:alnum:]]_*
1303 }

```

(End of definition for \c__bnvs_S_regex.)

`\c__bnvs_dot_R_regex` A possibly empty list of .*<short name>* items representing a path. Integers are allowed as well.

```

1304 \regex_const:Nn \c__bnvs_R_regex {
1305   \ur{c__bnvs_index_regex} | [[:alnum:]]_+
1306 }

```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_dot_R_regex` A possibly empty list of .*<short name>* items representing a path. Integers are allowed as well.

```

1307 \regex_const:Nn \c__bnvs_dot_R_regex {
1308   \. \ur{c__bnvs_R_regex}
1309 }

```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_A_reserved_Z_regex` Reserved words.

(End of definition for \c__bnvs_A_reserved_Z_regex.)

```

1310 \regex_const:Nn \c__bnvs_A_reserved_Z_regex {
1311   \A (?:_+ \d | _* [a-z] ) [_a-z0-9]* \Z
1312 }

```

`\c__bnvs_colons_regex` For ranges defined by a colon syntax. One catching group for more than one colon.

```

1313 \regex_const:Nn \c__bnvs_colons_regex { :(:+)? }

```

(End of definition for \c__bnvs_colons_regex.)

\c__bnvs_A_VAZLLZZL_Z_regex Used to parse named overlay specifications. V, A:Z, A::L on one side, :Z, :Z::L and ::L:Z on the other sides. Next are the capture groups. The first one is for the whole match.

(End of definition for \c__bnvs_A_VAZLLZZL_Z_regex.)

```

1314 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
1315   \A \s* (?:
      • 2 → V
      ( [^:]+? )
      • 3, 4, 5 → A : Z? or A :: L?
1317   | (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
      • 6, 7 → ::(L:Z)?
1318   | (?: :: \s* (?: ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
      • 8, 9 → :(Z::L)?
1319   | (?: : \s* (?: ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
1320   )
1321   \s* \Z
1322 }
```

6.9 End group setters

```

1323 \cs_new:Npn \BNVS_after_end_tl_put_right:cv #1 {
1324   \BNVS_after_end:n {
1325     \__bnvs_tl_put_right:cn { #1 }
1326   }
1327   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1328 }

1329 \cs_new:Npn \BNVS_end_tl_put_right:cv #1 #2 {
1330   \BNVS_after_end_tl_put_right:cv { #1 } { #2 }
1331   \BNVS_end:
1332 }

1333 \cs_new:Npn \BNVS_after_end_tl_gset:nnnv #1 #2 #3 {
1334   \BNVS_after_end:n {
1335     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
1336   }
1337   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1338 }

1339 \cs_new:Npn \BNVS_end_tl_gset:nnnv #1 #2 #3 #4 {
1340   \BNVS_after_end_tl_gset:nnnv { #1 } { #2 } { #3 } { #4 }
1341   \BNVS_end:
1342 }
```

```

1343 \cs_new:Npn \BNVS_after_end_tl_set:cv #1 {
1344   \BNVS_after_end:n {
1345     \__bnvs_tl_set:cn { #1 }
1346   }
1347   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1348 }

1349 \cs_new:Npn \BNVS_end_tl_set:cv {
1350   \BNVS_tl_set:ncv { \BNVS_end: }
1351 }

1352 \cs_new:Npn \BNVS_tl_set:ncv #1 #2 {
1353   \BNVS_tl_use:nv {
1354     #1
1355     \__bnvs_tl_set:cn { #2 }
1356   }
1357 }

1358 \cs_new:Npn \BNVS_tl_set:ncvcv #1 #2 #3 #4 {
1359   \BNVS_tl_set:ncv {
1360     \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1361   } { #4 }
1362 }

1363 \cs_new:Npn \BNVS_tl_set:ncvcvcv #1 #2 #3 #4 #5 #6 {
1364   \BNVS_tl_set:ncv {
1365     \BNVS_tl_set:ncv {
1366       \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1367     } { #4 } { #5 }
1368   } { #6 }
1369 }

1370 \cs_new:Npn \BNVS_after_end_int_set:cv #1 {
1371   \BNVS_after_end:n {
1372     \__bnvs_int_set:cn { #1 }
1373   }
1374   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1375 }

1376 \cs_new:Npn \BNVS_end_int_set:cv #1 #2 {
1377   \BNVS_after_end_int_set:cv { #1 } { #2 }
1378   \BNVS_end:
1379 }

```

6.10 beamer.cls interface

Work in progress.

```

1380 \RequirePackage{keyval}

1381 \define@key{beamerframe}{beanoves~id}[]{}
1382 \tl_set:Nx \l__bnvs_J_tl { #1 }
1383 }

```

```

1384 \bool_new:N \l__bnvs_in_frame_bool
1385 \bool_set_false:N \l__bnvs_in_frame_bool
1386 \AddToHook{env/beamer@frameslide/before}{
1387   \__bnvs_greset_call:
1388   \__bnvs_VS_gunset:
1389   \__bnvs_set_true:c { in_frame }
1390 }

1391 \AddToHook{env/beamer@frameslide/after}{
1392   \__bnvs_set_false:c { in_frame }
1393 }

```

6.11 Utilities

6.11.1 Split utilities

<code>__bnvs_if_regex_split:cncTF</code>	<code>__bnvs_if_regex_split:cncTF {<regex core>} {<expression>} {<seq core>}</code>
<code>__bnvs_if_regex_split:cnTF</code>	<code>{<yes:>} {<no:>}</code>

<code>__bnvs_if_regex_split:cnTF</code>	<code>{<regex core>} {<expression>} {<yes:>} {<no:>}</code>
--	---

These are shortcuts to `\regex_split:NnNTF` with the split sequence as last N argument.

```

1394 \BNVS_new_conditional:cpnn { if_regex_split:cnc } #1 #2 #3 { T, F, TF } {
1395   \BNVS_seq_use:nc {
1396     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1397   } { #3 } \prg_return_true: \prg_return_false:
1398 }

```

```

1399 \BNVS_new_conditional:cpnn { if_regex_split:cn } #1 #2 { T, F, TF } {
1400   \BNVS_seq_use:nc {
1401     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1402   } { split } \prg_return_true: \prg_return_false:
1403 }

```

<code>__bnvs_split_pop:</code>	<code>__bnvs_split_pop:</code>
<code>__bnvs_split_pop:c</code>	<code>__bnvs_split_pop:c {<var₁>}</code>
<code>__bnvs_split_pop:cc</code>	<code>__bnvs_split_pop:cc {<var₁>} {<var₂>}</code>
<code>__bnvs_split_pop:ccc</code>	<code>__bnvs_split_pop:ccc {<var₁>} {<var₂>} {<var₃>}</code>
<code>__bnvs_split_pop:cccc</code>	<code>__bnvs_split_pop:cccc {<var₁>} {<var₂>} {<var₃>} {<var₄>}</code>
<code>__bnvs_split_pop:ccccc</code>	<code>__bnvs_split_pop:ccccc {<var₁>} {<var₂>} {<var₃>} {<var₄>} {<var₅>}</code>

```

1404 \tl_new:N \l__bnvs_split_pop_tl
1405 \BNVS_new:cpn { split_pop: } {
1406   \seq_pop_left:NN \l__bnvs_split_seq \l__bnvs_split_pop_tl
1407 }

```

```

1408 \BNVS_new:cpn { split_pop:c } #1 {
1409   \BNVS_tl_use:nc {
1410     \seq_pop_left:NN \l__bnvs_split_seq
1411   } { #1 }
1412 }

```

```

1413 \BNVS_new:cpn { split_pop:cc } #1 {
1414   \__bnvs_split_pop:c { #1 }
1415   \__bnvs_split_pop:c
1416 }

1417 \BNVS_new:cpn { split_pop:ccc } #1 {
1418   \__bnvs_split_pop:c { #1 }
1419   \__bnvs_split_pop:cc
1420 }

1421 \BNVS_new:cpn { split_pop:cccc } #1 {
1422   \__bnvs_split_pop:c { #1 }
1423   \__bnvs_split_pop:ccc
1424 }

1425 \BNVS_new:cpn { split_pop:ccccc } #1 {
1426   \__bnvs_split_pop:c { #1 }
1427   \__bnvs_split_pop:cccc
1428 }

1429 \BNVS_new_conditional:cpnn { split_if_pop:c } #1 { T, F, TF } {
1430   \__bnvs_seq_pop_left:ccTF { split } { #1 } {
1431     \prg_return_true:
1432   } {
1433     \prg_return_false:
1434   }
1435 }

```

6.11.2 Match utilities

```

\__bnvs_match_if_once:NnTF \__bnvs_match_if_once:NnTF <regex variable> {<expression>} {<yes:>} {<no:>}
\__bnvs_match_if_once:NvTF \__bnvs_match_if_once:nnTF {<regex>} {<expression>} {<yes:>} {<no:>}
\__bnvs_match_if_once:nnTF

```

These are shortcuts to

- \regex_match_if_once:NnNTF with the match sequence as N argument
- \regex_match_if_once:nnNTF with the match sequence as N argument
- \regex_split:NnNTF with the split sequence as last N argument

```

1436 \BNVS_new_conditional:cpnn { if_extract_once:Ncn } #1 #2 #3 { T, F, TF } {
1437   \BNVS_use:ncn {
1438     \regex_extract_once:NnNTF #1 { #3 }
1439   } { #2 } { seq } \prg_return_true: \prg_return_false:
1440 }

```

\l__bnvs_match_seq Local storage for the match result.

(End of definition for \l__bnvs_match_seq.)

```

1441 \BNVS_new_conditional:cpnn { match_if_once:Nn } #1 #2 { T, F, TF } {
1442   \BNVS_use:ncn {
1443     \regex_extract_once:NnNTF #1 { #2 }
1444   } { match } { seq } \prg_return_true: \prg_return_false:
1445 }

1446 \BNVS_new_conditional:cpnn { if_extract_once:Ncv } #1 #2 #3 { T, F, TF } {
1447   \BNVS_seq_use:nc {
1448     \BNVS_tl_use:nv {
1449       \regex_extract_once:NnNTF #1
1450     } { #3 }
1451   } { #2 } \prg_return_true: \prg_return_false:
1452 }

1453 \BNVS_new_conditional:cpnn { match_if_once:Nv } #1 #2 { T, F, TF } {
1454   \BNVS_seq_use:nc {
1455     \BNVS_tl_use:nv {
1456       \regex_extract_once:NnNTF #1
1457     } { #2 }
1458   } { match } \prg_return_true: \prg_return_false:
1459 }

1460 \BNVS_new_conditional:cpnn { match_if_once:nn } #1 #2 { T, F, TF } {
1461   \BNVS_seq_use:nc {
1462     \regex_extract_once:nnNTF { #1 } { #2 }
1463   } { match } \prg_return_true: \prg_return_false:
1464 }

1465 \BNVS_new_conditional:cpnn { match_if_pop_left:c } #1 { T, F, TF } {
1466   \BNVS_tl_use:nc {
1467     \BNVS_seq_use:Nc \seq_pop_left:NNTF { match }
1468   } { #1 } {
1469     \prg_return_true:
1470   } {
1471     \prg_return_false:
1472   }
1473 }

```

__bnvs_match_pop:	__bnvs_match_pop:	
__bnvs_match_pop:c	__bnvs_match_pop:c	{\langle var ₁ \rangle}
__bnvs_match_pop:cc	__bnvs_match_pop:cc	{\langle var ₁ \rangle} {\langle var ₂ \rangle}
__bnvs_match_pop:ccc	__bnvs_match_pop:ccc	{\langle var ₁ \rangle} {\langle var ₂ \rangle} {\langle var ₃ \rangle}
__bnvs_match_pop:cccc	__bnvs_match_pop:cccc	{\langle var ₁ \rangle} {\langle var ₂ \rangle} {\langle var ₃ \rangle} {\langle var ₄ \rangle}
__bnvs_match_pop:ccccc	__bnvs_match_pop:ccccc	{\langle var ₁ \rangle} {\langle var ₂ \rangle} {\langle var ₃ \rangle} {\langle var ₄ \rangle} {\langle var ₅ \rangle}

```

1474 \tl_new:N \l__bnvs_match_pop_tl
1475 \BNVS_new:cpn { match_pop: } {
1476   \seq_pop_left:NN \l__bnvs_match_seq \l__bnvs_match_pop_tl
1477 }

1478 \BNVS_new:cpn { match_pop:c } #1 {
1479   \BNVS_tl_use:nc {
1480     \seq_pop_left:NN \l__bnvs_match_seq
1481   } { #1 }
1482 }

```

```

1483 \BNVS_new:cpn { match_pop:cc } #1 {
1484   \__bnvs_match_pop:c { #1 }
1485   \__bnvs_match_pop:c
1486 }

1487 \BNVS_new:cpn { match_pop:ccc } #1 {
1488   \__bnvs_match_pop:c { #1 }
1489   \__bnvs_match_pop:cc
1490 }

1491 \BNVS_new:cpn { match_pop:cccc } #1 {
1492   \__bnvs_match_pop:c { #1 }
1493   \__bnvs_match_pop:ccc
1494 }

1495 \BNVS_new:cpn { match_pop:ccccc } #1 {
1496   \__bnvs_match_pop:c { #1 }
1497   \__bnvs_match_pop:cccc
1498 }

```

6.12 Scanner

We use the same parser for definitions and queries. We do not use any key-value facility built into latex.

6.12.1 Grammar

The description of the various symbols used is inspired by regular expression syntax, except that the dot character keeps its literal meaning and double quotes (") are used to escape some literals. This is a pragmatic grammar used for parsing, not all the expressions do have a real meaning.

```

1  <blnc>   = ( <call> | <rslv> | <rnd> | <sqr> | <crl> | <other> ) *
2  <other>  = [^()]"["{,} *
3  <assign> = ( ( <IKSR> | <P> ) ( [!?] )? "=" )+ <blnc>
4  <IKSR>   = \s * ( <S> ? ! )? <S> ( "." <R> ) * \s *
5  <P>      = \s * [ <alnum>_ ] + ( "." <R> ) * \s *
6  <R>      = <index> | [ <alnum>_ ] +
7  <S>      = ( ( <lower> | _ )? <alnum> * )?
8           <upper>
9           ( <alnum> | _ ) *
10 <index>  = [ -+ \s ] * <digit> +
11 <VAZL>   = <blnc>
12         | <blnc> : <blnc>? ( :: <blnc> ? ) ?
13         | <blnc> :: <blnc>? ( : <blnc> ? ) ?
14 <list>   = <VAZL> ( , <VAZL> ) *
15 <call>   = [ <alnum>_ ] [ <digit> <alnum>_ "." ] * <rnd>
16 <rslv>   = "?" <rnd>
17 <rnd>    = "(" <list> ")"
18 <sqr>    = "[" <list> "]"
19 <crl>    = "{" <list> "}"

```

6.12.2 Start

`\_bnvs_scan_start:` `\_bnvs_scan_start:`

Various management tasks.

```
1499 \BNVS_new:cpn { scan_start: } {
1500   \_bnvs_tl_clear:c S
1501 }
```

6.12.3 Storage

There are different policies to store the intermediate results of a scanning operation. One possibility is to feed a global variable as scanning occurs. As you cannot use unbalanced braces in function arguments, we can use other characters than usual “\”, “{” and “}” before we rescan the final result with an appropriate catcode regime. Another possibility is to heavily use T_EX groups to store intermediate results in local variables and only consider properly balanced material. At the end of the group, the scanned material must be forwarded to the group owner. The latter is used here.

`\BNVS_scanned:n` `\BNVS_scanned:n {⟨string⟩}`

The default implementation append the `⟨string⟩` to the `S` variable. This is sometimes redefined to store the argument in a different local variable.

```
1502 \cs_new:Npn \BNVS_scanned:n {
1503   \_bnvs_tl_put_right:cn S
```

⊘ There must be absolutely no code here. No hooking allowed either.

```
1504 }
```

6.12.4 Branch with elementary functions

In the sequel, `⟨no code:w⟩` is expected to read the `⟨input stream⟩` until a `\q_stop` token that is gobbled. The `_scan_` in function names is a shortcut to indicate a peek and remove operation.

`\_bnvs_scan_q_stop:Fw` `\_bnvs_scan_q_stop:Fw {⟨no code:w⟩} ⟨input stream⟩`

On scanning a `\q_stop` from the head of the `⟨input stream⟩`, calls the `\BNVS_scan_stopped:` branch function, otherwise execute `⟨no code:w⟩`.

```
1505 \BNVS_new:cpn { scan_q_stop:Fw } {
1506   \peek_meaning_remove:NTF \q_stop {
1507     \BNVS_scan_stopped:
1508   }
```

⊘ There must be absolutely no code here. No hooking allowed either.

```
1509 }
```

`\_bnvs_scan_comma:Fw` `\_bnvs_scan_comma:Fw` $\langle no\ code:w \rangle$ $\langle input\ stream \rangle$

On scanning a comma from the head of the $\langle input\ stream \rangle$, calls `\BNVS_scanned_comma:w` branch function, otherwise execute $\langle no\ code:w \rangle$.

```

1510 \BNVS_new:cpn { scan_comma:Fw } #1 {
1511   \peek_remove_spaces:n {
1512     \peek_charcode_remove:NTF , {
1513       \peek_remove_spaces:n { \BNVS_scanned_comma:w }
1514     } {
1515       #1
1516     }
1517   }
1518 }
```

D

`\_bnvs_scan_colon:Fw` `\_bnvs_scan_colon:Fw` $\{ \langle no\ code:w \rangle \}$ $\langle input\ stream \rangle$

On scanning a standalone colon from the $\langle input\ stream \rangle$, calls the `\BNVS_scanned_one_colon:w` branch function. On scanning multiple colons from the $\langle input\ stream \rangle$, calls the `\BNVS_scanned_colons:w` branch function. In other cases, execute the $\langle no\ code:w \rangle$.

```

1519 \BNVS_new:cpn { scan_colon:Fw } #1 {
1520   \peek_remove_spaces:n {
1521     \peek_charcode_remove:NTF : {
1522       \peek_charcode_remove:NTF : {
1523         \peek_remove_spaces:n { \BNVS_scanned_colons:w }
1524       } {
1525         \peek_remove_spaces:n { \BNVS_scanned_one_colon:w }
1526       }
1527     } {
1528       #1
1529     }
1530   }
1531 }

1532 \cs_set:Npn \BNVS_scanned_comma:w {
1533   \BNVS_tl_use:Nv \BNVS_end_scan_out:n S
1534   \end{macrocode}
1535 % \begin{BNVS.gobble}
1536 \<!*final>
1537   \BNVS_DEBUG_begin:N \BNVS_scanned_comma:w
1538 \</!final>
1539 \<!*debug>
1540 % \end{BNVS.gobble}
1541 %   \begin{macrocode}
1542   \BNVS_begin:
1543   \_bnvs_tl_clear:c S
1544 }
```

6.12.5 Assignments and identifiers

`\c__bnvs_IKSRMA_regex` To scan assignment or standalone overlay set identifier.

```

1545 \regex_const:Nn \c__bnvs_IKSRMA_regex {
      1: the frame <id>
      2: the exclamation mark
1546   (? : ( \ur{c__bnvs_S_regex} )? ( ! ) )?
      3: The short name.
1547   ( \ur{c__bnvs_S_regex} )
      4: the remaining part of the path, if any.
1548   ( \ur{c__bnvs_dot_R_regex} * )
      5: the assignement modifier.
1549   (? : ( [-+/*?] )?
      6: the remaining part of the path, if any.
1550   ( = ) )?
1551   \s*
1552 }
```

(End of definition for \c__bnvs_IKSRMA_regex.)

6.12.6 Delimiters

`\__bnvs_scan_delimited:NNnFw` `\__bnvs_scan_delimited:NNnFw` `<open>` `<close>` `{<done:nw>}` `{<alt:w>}`

If we scan the `<open>` delimiter, we open a group that will be closed on scanning the next unbalanced closing delimiter. Only `<close>` raises no error. The `\BNVS_closed:nw` branch is then executed with the closing delimiter as argument. If the scanning is stopped prematurely, then the `\BNVS_stopped:` branch is executed, after an error is raised. Uses `\__bnvs_scan_blnc:w` to scan balanced text inside the delimiters. Otherwise the `<alt:w>` branch is executed.

```

1553 \BNVS_new:cpn { scan_delimited:NNnFw } #1 #2 #3 {
1554   \peek_charcode_remove:NTF #1 {
1555     \BNVS_begin:
1556     \__bnvs_tl_clear_all:n { S T }
1557     \cs_set:Npn \BNVS_scan_out:n {
1558       \__bnvs_tl_put_right:cn T
1559     }
1560     \cs_set:Npn \BNVS_scan_closed:Nw ##1 {
1561       \__bnvs_tl_put_right:ce S {
1562         \exp_not:N \:n { \BNVS_tl_use:Nv \exp_not:n T }
1563       }
1564       \tl_if_eq:nnF { #2 } { ##1 } {
1565         \BNVS_error:n { Unexpected~`##1'~instead~of~`#2'. }
1566       }

```

```

1567 \cs_set:Npn \BNVS:n #####1 { #3 }
1568 \BNVS_tl_use:nv {
1569   \BNVS_end:
1570   \BNVS:n
1571 } T
1572 \BNVS_scan_closed:Nw ##1
1573 }
1574 \cs_set:Npn \BNVS_scan_stopped: {
1575   \BNVS_error:n { Missing~`#2'. }
1576   \_bnvs_tl_put_right:ce S {
1577     \exp_not:N \:n { \BNVS_tl_use:Nv \exp_not:n T }
1578   }
1579   \cs_set:Npn \BNVS:n #####1 { #3 }
1580   \BNVS_tl_use:nv {
1581     \BNVS_end:
1582     \BNVS:n
1583   } T
1584   \BNVS_scan_stopped:
1585 }
1586 \cs_set:Npn \BNVS_scanned_comma: {
1587   \_bnvs_tl_put_right:ce S {
1588     \exp_not:N \:n { \BNVS_tl_use:Nv \exp_not:n T }
1589   }
1590 }
1591 \__bnvs_scan_blnc:w
1592 }

```

⊘ There must be absolutely no code here. No hooking allowed either.

```

1593 }

```

At the top level, `\BNVS_scan_stopped:` branch function is a do nothing operation.

```

1594 \cs_set:Npn \BNVS_scan_stopped: {}

```

At the top level, `\BNVS_scan_out:n` helper raises an error.

```

1595 \cs_set:Npn \BNVS_scan_out:n #1 {
1596   \BNVS_error:n { Unexpected~call~to~\tl_to_str:N \BNVS_scan_out:n }
1597 }

```

Utility function to store the argument enclosed within on pair of curly braces, after a $\TeX$ group is closed.

```

1598 \cs_set:Npn \BNVS_end_scan_out:n #1 {
1599   \BNVS_end:
1600   \BNVS_scan_out:n { { #1 } }
1601 }

```

At the top level, `\BNVS_scan_closed:Nw` branch function raises an error.

```

1602 \cs_set:Npn \BNVS_scan_closed:Nw #1 {
1603   \BNVS_error:n { Unexpected~call~to~\tl_to_str:N \BNVS_scan_closed:Nw }
1604 }

```